



Python程序设计：面向对象编程



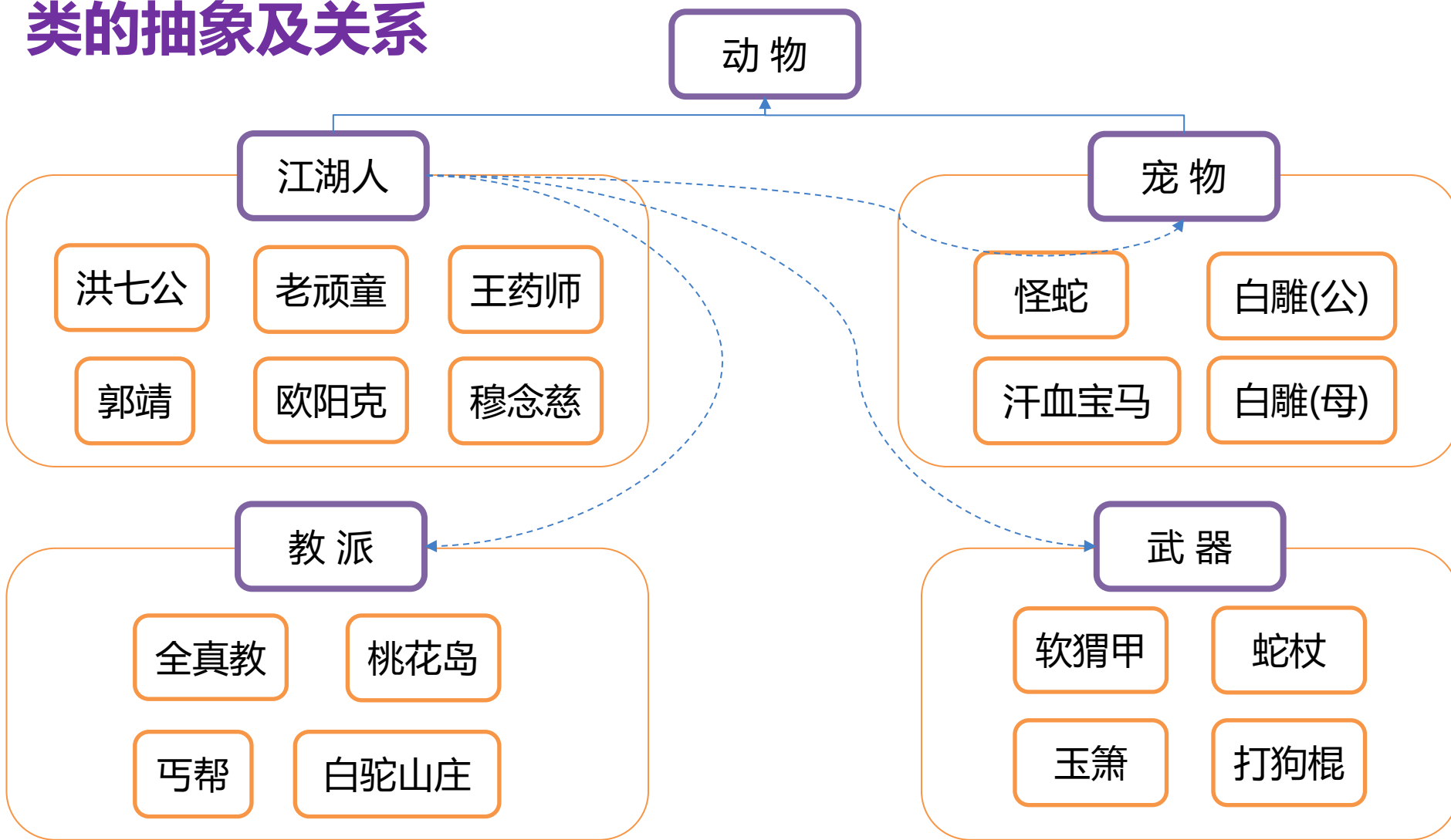
南开大学软件学院

王斌辉

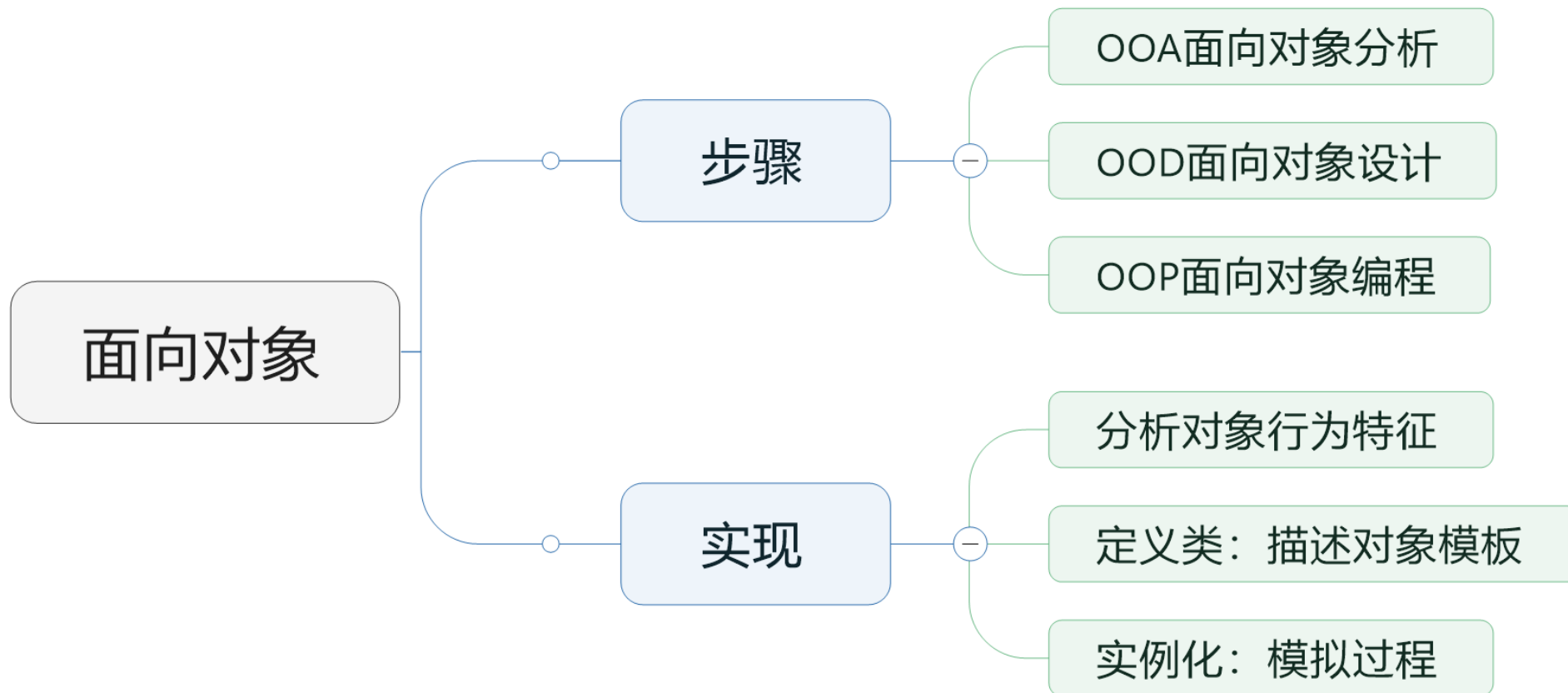
■ 函数式编程与面向对象编程

- 函数式编程风格的优点：
 - [形式证明](#)
 - 模块化（把一个问题分解成多个小的方面）
 - 组合性（函数重复使用，甚至可以组织已有的函数来组成新的程序）
 - 易于调试和测试（调试：函数小且明确；测试：每个函数皆是单元测试的潜在目标）
- 函数式编程希望尽可能地消除状态变化，只和流经函数的数据打交道
- 面向对象编程中，对象如同一颗小胶囊，包裹着内部状态和修改这个内部状态的一组调用方法，以及由正确的状态变化所构成的程序
- 在 Python 中，最好把两种编程方式结合起来，在各类应用（诸如电子邮件信息，事务处理）中编写接受和返回对象实例的函数。

■ 类的抽象及关系



■ 类的理解与使用



- 封装、继承、多态

■ 如何定义类

- class 关键字

```
class Human:  
    pass
```

```
h = Human()
```

- h是类Human的实例(instance)/对象 (object)
- Human() 称为实例化

■ 如何定义类

— 经典类与新式类

- **Python 2.x中默认都是经典类**，只有显式继承object类才是新式类
- **Python 3.x中皆为新式类，经典类被移除**，类默认继承自object且不必显式继承
- object类内建了很多原始的属性和方法，用户自定义的类默认会继承这些属性和方法，可使用函数dir()查看。object虽内建了很多方法，但实际开发中经常重写这些内置方法后才会使用它们
- 新式类是Python2.2为了统一类和实例（type/class unification）引入的。新式类中类与类型已经统一，实例的类型是创建该实例的类，通常和该实例的__class__值相同，而非Python2.x中的instance实例类型(旧式类中，类名与type无关)。如x为旧式类实例，则x.__class__定义了x的类名，但type(x)总返回<type 'instance'>(即所有旧式类的实例，均通过单一的instance内建类生成)；如x为新式类的实例，则type(x)和x.__class__的结果相同（不完全保证，因为新式类的实例__class__允许被用户覆盖）

■ 如何定义类

- 类的属性和方法

```
class Student:  
    university = "Nankai University"  
  
    def take_class(self):  
        print("Having Python class...")
```

■ 如何定义类

— 类的属性和方法

- 实例可以访问类的属性
- 类无法访问实例方法
- 类无法访问实例属性

TypeError:
Student.take_class()
missing 1 required
positional argument: 'self'

```
class Student:
    university = "Nankai University"
```

```
    def take_class(self):
        print("Having Python class...")
```

```
gj = Student()
print(Student.university)
print(gj.university)
gj.take_class()
```

```
Student.take_class()
```

```
Student.take_class(gj)
hr = Student()
print("gj:", id(gj))
print("hr:", id(hr))
print(gj is hr)
```

```
gj: 2123573034480
hr: 2123573034336
False
```

■ 如何定义类

— 类的初始化

```
class Student:
    university = "Nankai University"

    def __init__(self, name, school):
        self.name = name
        self.school = school
```

```
    def take_class(self):
        print(f"{self.name} is having {self.school}'s class...")
```

```
gj = Student("Guo Jing", "G School")
hr = Student("Huang Rong", "T School")
gj.take_class()
hr.take_class()
```

- `__init__()` 是初始化方法, 用于实例的初始化
- `self.name` 等是实例属性

■ 如何定义类

— 类的初始化

```
class Student:
    university = "Nankai University"

    def __init__(self, name, school):
        self.name = name
        self.school = school

    def take_class(self):
        print(f"{self.name} is having {self.school}'s class...")
```

```
gj = Student("Guo Jing", "G School")
hr = Student("Huang Rong", "T School")
gj.take_class()
hr.take_class()
```

- 实例能访问其类属性
- 类无法访问实例属性

```
gj.unique_skill = "Nine Yin Kongfu"
print(gj.unique_skill)
print(Student.name)
print(Student.unique_skill)
print(hr.unique_skill)
print(gj.__dict__)
print(hr.__dict__)
```

```
{'name': 'Guo Jing', 'school': 'G School', 'unique_skill': 'Nine Yin Kongfu'}
{'name': 'Huang Rong', 'school': 'T School'}
```

■ 如何定义类

— 属性的查找顺序和修改

```
class Student:
    capacity = 10

    def __init__(self, name, school):
        self.name = name
        self.school = school
        print(self.capacity)

    def practice(self):
        print(self.capacity)

gj = Student("Guo Jing", "G School")
print(gj.capacity)
gj.capacity = 100
Student.capacity = 60
print(gj.capacity)
print(Student.capacity)
hr = Student("Huang Rong", "T School")
print(hr.capacity)
```



■ 如何定义类

- 定义类的三种方法
 - 实例方法
 - 类的方法（类方法）
 - 静态方法

■ 如何定义类

— 实例方法

- 实例方法：没有任何装饰器，有默认self的普通函数
- 类调用实例方法(详见方法绑定内容)，需要先传入实例作为参数

`Student.practice(hr)`

```
class Student:
    capacity = 10

    def __init__(self, name, school):
        self.name = name
        self.school = school
        print(self.capacity)

    def practice(self):
        print(self.capacity)

gj = Student("Guo Jing", "G School")
print(gj.capacity)
gj.capacity = 100
Student.capacity = 60
print(gj.capacity)
print(Student.capacity)
hr = Student("Huang Rong", "T School")
print(hr.capacity)
```

■ 如何定义类

— 类方法

- 类方法：有 classmethod 装饰的函数
- 类方法：既可以被类调用，也可以被实例调用

```
class Student:  
    capacity = 10
```

```
    def __init__(self, name, school):  
        self.name = name  
        self.school = school  
        print(self.capacity)
```

```
    def practice(self):  
        print(self.capacity)
```

```
    @classmethod  
    def show_capacity(cls):  
        print(f"The class' is {cls.capacity}")
```

```
    show_capacity = classmethod(show_capacity)
```

```
gj = Student("Guo Jing", "G School")  
Student.show_capacity = classmethod(Student.show_capacity)
```

■ 如何定义类

— 静态方法

- 静态方法：有 `staticmethod` 装饰的函数
- 静态方法无法通过 `self` 或者 `cls` 访问到实例或类中的属性
- 静态方法可以被实例或类调用
- 静态方法作用：添加针对类的一些规范、约束等，如针对学生的行为规则、上课规则等。

```
class Student:  
    capacity = 10
```

```
    def __init__(self, name, school):  
        self.name = name  
        self.school = school  
        print(self.capacity)
```

```
    def practice(self):  
        print(self.capacity)
```

```
@staticmethod  
def show_motto():  
    print(f"Public Interests & Capability")
```

```
gj = Student("Guo Jing", "G School")  
gj.show_motto()  
Student.show_motto()
```

■ 类中的绑定方法

— 实例的绑定方法

```
class Student:
    capacity = 10

    def __init__(self, name, school):
        self.name = name
        self.school = school
        print(self.capacity)

    def practice(self):
        print(self.capacity)

gj = Student("Guo Jing", "G School")
print(gj.practice)
```

```
<bound method Student.practice of <__main__.Student object at 0x000002039F663A90>>
```

■ 类中的绑定方法

— 实例的绑定方法

- 类中的方法，默认都是绑定给实例使用的
- 绑定方法都有自动传值的功能，传递的值，就是对象本身
- 类调用实例方法，实例方法仅被视为函数，无自动传值这一功能

```
class Student:
    capacity = 10

    def __init__(self, name, school):
        self.name = name
        self.school = school
        print(self.capacity)

    def practice(self):
        print(self.capacity)

gj = Student("Guo Jing", "G School")
print(gj.practice)
gj.practice()
print(Student.practice)
```

```
<bound method Student.practice of <__main__.Student object at 0x000002039F663A90>>
<function Student.practice at 0x000002039F65D480>
```

■ 类中的绑定方法

— 实例的绑定方法

- 类中的方法或函数，默认都是绑定给实例使用的
- 绑定方法都有自动传值的功能，传递的值，就是对象本身
- 类调用实例方法，实例方法仅被视为函数，无自动传值这一功能
- 类想调用实例方法，必须遵循函数的参数规则，有几个参数，就必须传递几个参数

```
class Student:
    capacity = 10

    def __init__(self, name, school):
        self.name = name
        self.school = school
        print(self.capacity)

    def practice(self):
        print(self.capacity)

gj = Student("Guo Jing", "G School")
print(gj.practice)
gj.practice()
print(Student.practice)
Student.practice(gj)
```

■ 类中的绑定方法

— 类的绑定方法

- 通过classmethod装饰器，将绑定给实例的方法，绑定到了类
- 如果一个方法绑定谁，在调用该函数时，将自动该调用者当作第一个参数传递到函数中
- 无论类还是实例调用类的绑定方法，都会自动将类当作参数传递

```
class Student:
    capacity = 10

    def __init__(self, name, school):
        self.name = name
        self.school = school
        print(self.capacity)

    def practice(self):
        print(self.capacity)

    @classmethod
    def show_capacity(cls):
        print(f"The class' is {cls.capacity}")

gj = Student("Guo Jing", "G School")
print(Student.show_capacity)
print(gj.show_capacity)
```

```
<bound method Student.show_capacity of <class '__main__.Student'>>
<bound method Student.show_capacity of <class '__main__.Student'>>
```

■ 类的非绑定方法

— 类的非绑定方法

- 通过staticmethod装饰器，可以解除绑定关系，将一个类中的方法，变为一个普通函数
- 静态方法中参数传递跟普通函数相同，无需考虑自动传参等问题

```
<function Student.show_motto at 0x00000274FBE0D630>  
<function Student.show_motto at 0x00000274FBE0D630>
```

```
class Student:  
    capacity = 10  
  
    def __init__(self, name, school):  
        self.name = name  
        self.school = school  
        print(self.capacity)  
  
    def practice(self):  
        print(self.capacity)  
  
    @staticmethod  
    def show_motto():  
        print(f"Public Interests & Capability")  
  
gj = Student("Guo Jing", "G School")  
print(Student.show_motto)  
print(gj.show_motto)
```

■ 方法与函数

- 可使用 type() 验证
- 普通函数(未定义在类里)是函数(function)
- 静态方法是函数(function)
- 实例方法和类方法, 都是方法(method)
- 方法是一种和对象(实例或者类)绑定后的特殊函数

```
class Student:  
    capacity = 10
```

```
def __init__(self, name, school):  
    self.name = name  
    self.school = school  
    print(self.capacity)
```

```
def practice(self):  
    print(self.capacity)
```

```
@classmethod  
def show_capacity(cls):  
    print(f"The class' is {cls.capacity}")
```

```
@staticmethod  
def show_motto():  
    print(f"Public Interests & Capability")
```

```
def func(): pass  
print(type(func))  
  
<class 'function'>
```

```
gj = Student("Guo Jing", "G School")  
print(type(gj.show_motto))  
print(type(gj.practice))  
print(type(gj.show_capacity))
```

```
<class 'function'>  
<class 'method'>  
<class 'method'>
```

■ 私有属性与私有方法

— 下划线

- 单前导下划线: `_var` PEP 8 规定, 有单前导下划线, 则该属性应该仅供内部访问
- 单末尾下划线: `var_`
- 双前导下划线: `__var`
- 双前导和末尾下划线: `__var__`
- 单下划线: `_`
- 数字中的下划线: `x = 1_000_000`

■ 类的私有属性

- 使用两个下划线开头，如__var，来声明该属性为私有属性
- 私有属性不能在类的外部被直接访问
- 实例方法，使用self.__var调用实例私有属性
- 类方法中，使用cls.__var调用类的私有属性
- 在静态方法中，不太方便调用实例的私有属性，类的私有属性可以使用类名调用

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.__age = age

    def is_adult(self):
        return self.__age >= 18
```

```
xh = Person(name="小红", age=27)
print(xh.is_adult())
xh.__age = 17
print(xh.__age)
print(xh.__dict__)
```

True

17

```
{'name': '小红', '_Person__age': 27, '__age': 17}
```

■ 私有属性的内外调用

```
class JustCounter:
    __secret_count = 0
    public_count = 0

    def count(self):
        self.__secret_count += 1
        self.public_count += 1
        print(self.__secret_count)
```

```
counter = JustCounter()
1 counter.count()
2 counter.count()
2 print(counter.public_count)
2 print(counter.__JustCounter__secret_count)
```

■ 私有方法

- 使用两个下划线开头，如__method()，来声明该方法为私有方法
- 私有方法不能在类的外部被直接调用

```
class Student:
    capacity = 10

    def __init__(self, name, school):
        self.name = name
        self.__school = school
        print(self.capacity)

    def practice(self):
        print(self.capacity)

    def __practice(self):
        print(f"in {self.__school}")

    @classmethod
    def show_capacity(cls):
        print(f"The class' is {cls.capacity}")
```

```
gj = Student("Guo Jing", "G School")
gj.practice()
gj.__practice()
```

AttributeError: 'Student' object
has no attribute '__practice'.

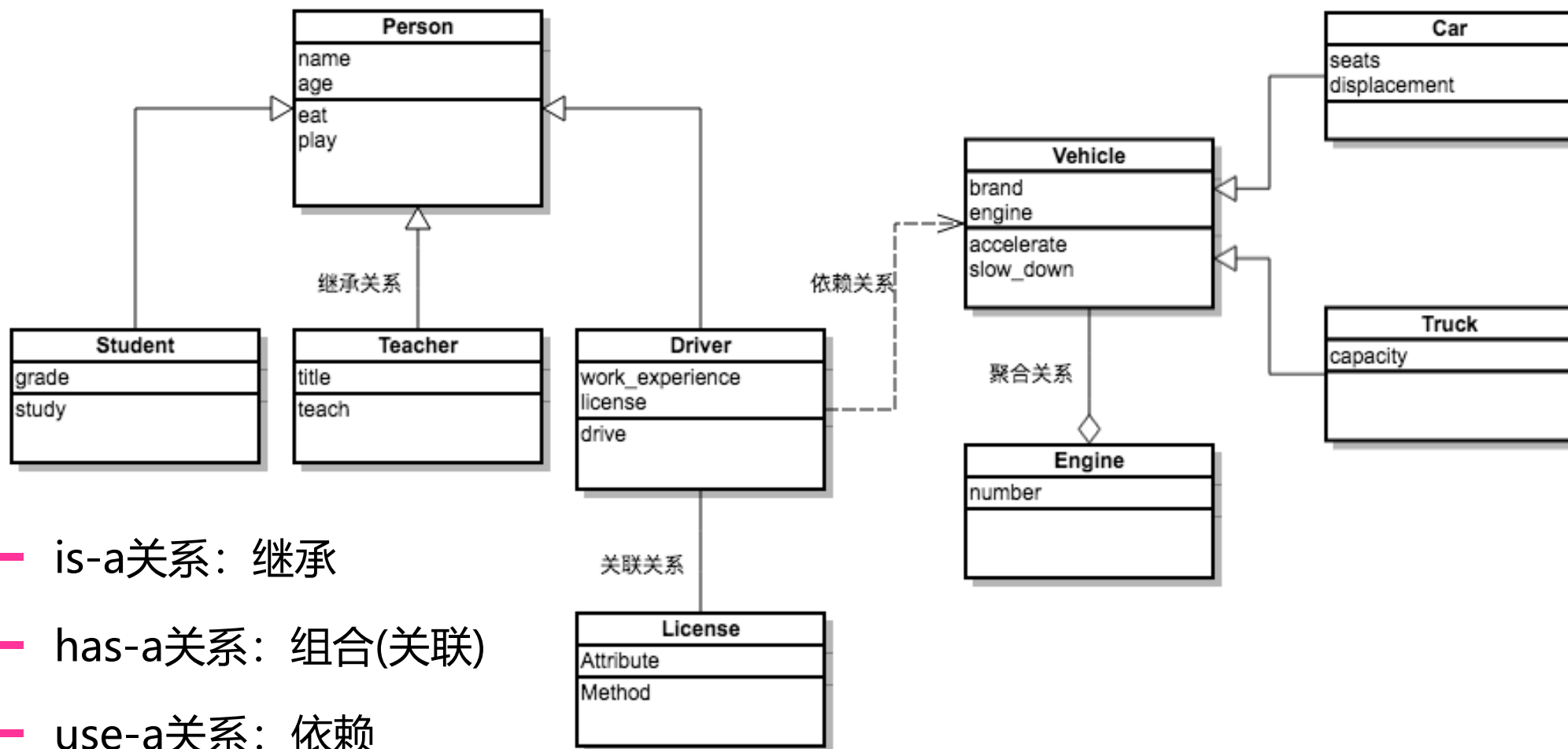
■ 类的有限封装

— Python的私有属性和方法的外部访问

类的外部外部可以通过：_类名_属性名 或者 _类名_方法名 来直接访问

类中的私有属性和私有方法

■ 类之间的关系



- is-a关系: 继承
- has-a关系: 组合(关联)
- use-a关系: 依赖

■ 类之间的关系

- use-a关系：依赖

```
class People:

    def __init__(self, name):
        self.name = name

    def open(self, frig):
        frig.open_door(self)

    def close(self, frig):
        frig.close_door(self)

class Refrigerator:

    def __init__(self, name):
        self.name = name

    def open_door(self, p):
        print(f"{p.name}打开冰箱")

    def close_door(self, p):
        print(f"{p.name}关闭冰箱")
```

```
gj = People("郭靖")
aux = Refrigerator("奥克斯")
gj.open(aux)
gj.close(aux)
```

■ 类之间的关系

- has-a关系: 组合(关联)

```
class Driver:
    def __init__(self, work_experience, dri_lic):
        self.work_experience = work_experience
        self.dri_lic = dri_lic

    def drive(self, veh):
        pass
```

```
class License:
    def __init__(self, name, lic_num):
        self.name = name
        self.lic_num = lic_num
```

```
lic = License("Guo Jing", 9988)
d = Driver(12, lic)
```



■ 类的继承 (Inheritance)

```
class People:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def speak(self):
        print(f"{self.name} 说: 我{self.age}岁。")

class Student:
    def __init__(self, name, age, weight):
        self.name = name
        self.age = age
        self.weight = weight

    def speak(self):
        print(f"{self.name} 说: 我{self.age}岁。")
```



■ 类的继承

- 基类(或父类) :
被继承的类
- 派生类(或子类):
继承的类

```
class People:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def speak(self):
        print(f"{self.name} 说: 我{self.age}岁。")
```

```
class Student(People):
    def __init__(self, name, age, weight, grade):
        People.__init__(self, name, age)
        self.weight = weight
        self.grade = grade
```

```
xm = Student(name="小明", age=10, weight=50, grade="三年级")
xm.speak()
```

■ 类的继承

— 方法的重写

```
class People:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def speak(self):
        print(f"{self.name} 说：我{self.age}岁。")
```

```
class Student(People):
    def __init__(self, name, age, weight, grade):
        People.__init__(self, name, age)
        self.weight = weight
        self.grade = grade

    def speak(self):
        print(f"{self.name} 说：我{self.age}岁了，我在读{self.grade}")
```

■ 类的继承 (Inheritance)

— 多继承

```
class O: pass

class D(O): pass

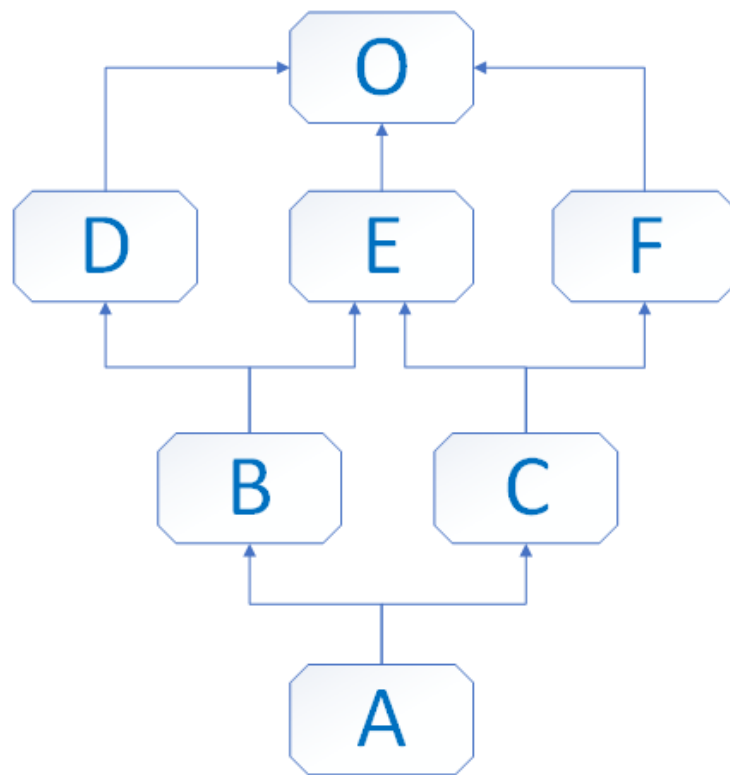
class E(O): pass

class F(O): pass

class B(D, E): pass

class C(E, F): pass

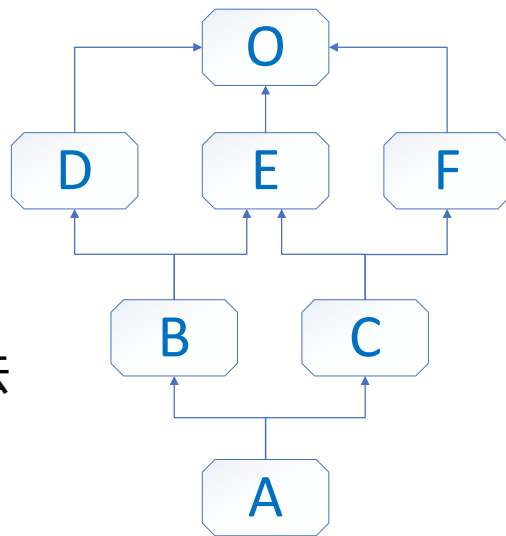
class A(B, C): pass
```



■ 类的继承 (Inheritance)

— MRO(Method Resolution Order)

- 使用 `__mro__` 来查询 `print(A.__mro__)`
- Python2.3之前, MRO基于深度优先搜索算法
- Python2.3起使用新算法: C3算法
- C3算法最早源于Lisp, Python为解决原基于深度优先搜索算法不能满足本地优先级和单调性的问题, 而采用该算法
 - 本地优先级: 指声明时父类的顺序, 如`C(A, B)`, 当访问C类对象属性时, 应根据声明顺序, 优先查找A类, 再查找B类
 - 单调性: 在C的继承顺序中, 如A在B前, 则C的所有子类, 也须满足该顺序
- 如右侧代码示例: Python2.3之前的G的继承顺序是: `G->E->F->O`, 但根据C3算法代码将报错



TypeError: Cannot create a consistent method resolution order (MRO) for bases F, E

```
class F:  
    pass
```

```
class E(F):  
    pass
```

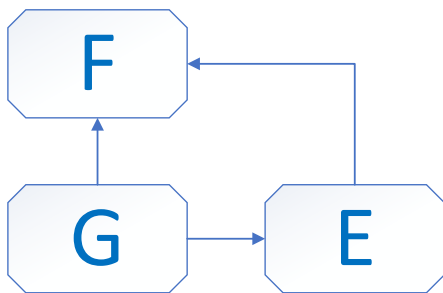
```
class G(F, E):  
    pass
```

```
print(G.__mro__)
```

■ 类的继承 (Inheritance)

— MRO(Method Resolution Order)

- C3算法可以通过左优先的拓扑算法来快速得到类的继承关系
- 但因其无法识别一些不满足本地优先级和单调性条件的继承关系，因此存在一定缺陷，如下边代码：



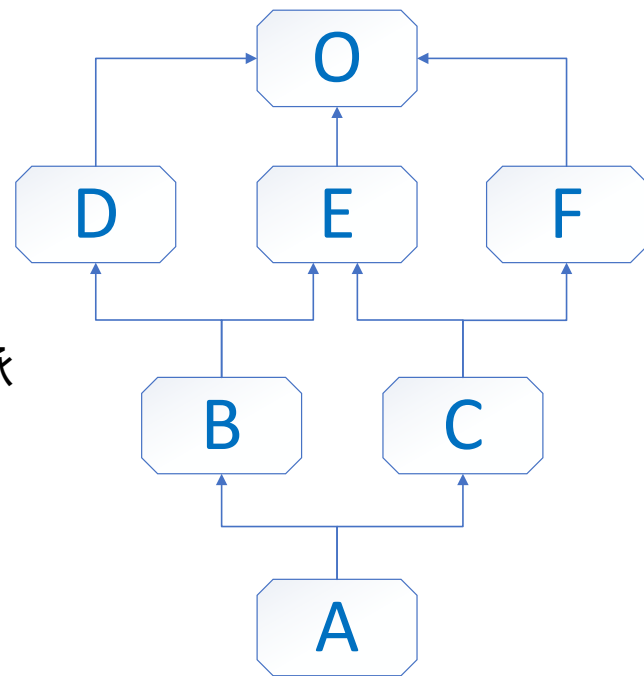
左优先的拓扑算法无法识别继承缺陷

```
class F:
    pass

class E(F):
    pass

class G(F, E):
    pass

print(G.__mro__)
```



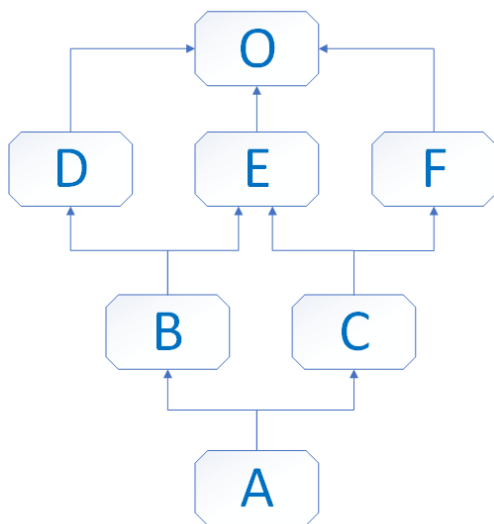
■ 类的继承 (Inheritance)

— C3算法：基于Merge推导

- ① 检查第一个列表的头元素记作 H
- ② 若 H出现在其它列表的头部，或未出现在其它列表中，则将其输出，并将其从所有列表中删除，然后回到步骤1；否则，取出下一个列表的头元素记作 H，继续该步骤
- ③ 重复上述步骤，直至列表为空或者不能再找出可以输出的元素。如果是前一种情况，则算法结束；如果是后一种情况，说明无法构建继承关系，Python 会抛出异常

■ 类的继承

— 基于Merge推导



```
mro(A) = mro( A(B,C) )
```

```
原式= [A] + merge( mro(B),mro(C),[B,C] )
```

```

o(B) = mro( B(D,E) )
      = [B] + merge( mro(D), mro(E), [D,E] ) # 多继承
      = [B] + merge( [D,O] , [E,O] , [D,E] ) # 单继承mro(D(O))=[D,O]
      = [B,D] + merge( [O] , [E,O] , [E] ) # 拿出并删除D
      = [B,D,E] + merge([O] , [O])
      = [B,D,E,O]

```

```

mro(C) = mro( C(E,F) )
        = [C] + merge( mro(E), mro(F), [E,F] )
        = [C] + merge( [E,O] , [F,O] , [E,F] )
        = [C,E] + merge( [O] , [F,O] , [F] ) # 跳过O, 拿出并删除
        = [C,E,F] + merge([O] , [O])
        = [C,E,F,O]

```

```

原式= [A] + merge( [B,D,E,O], [C,E,F,O], [B,C])
      = [A,B] + merge( [D,E,O], [C,E,F,O], [C])
      = [A,B,D] + merge( [E,O], [C,E,F,O], [C]) # 跳过E
      = [A,B,D,C] + merge([E,O], [E,F,O])
      = [A,B,D,C,E] + merge([O], [F,O]) # 跳过O
      = [A,B,D,C,E,F] + merge([O], [O])
      = [A,B,D,C,E,F,O]

```

■ 类的继承: 方法的重写

- 使用类名调用父类方法的问题

```
class Base:
    def __init__(self):
        print("enter Base")
        print("leave Base")
```

```
class A(Base):
    def __init__(self):
        print("enter A")
        Base.__init__(self)
        print("leave A")
```

```
class B(Base):
    def __init__(self):
        print("enter B")
        Base.__init__(self)
        print("leave B")
```

```
class C(A, B):
    def __init__(self):
        print("enter C")
        A.__init__(self)
        B.__init__(self)
        print("leave C")
```

```
c = C()
```

```
enter C
enter A
enter Base
leave Base
leave A
enter B
enter Base
leave Base
leave B
leave C
```

■ 类的继承 (Inheritance)

— super()

- 子类中定义了与父类同名的方法，则子类的方法会覆盖父类方法
- super() 语法: super(type[, object-or-type])
 - type -- 类
 - object-or-type – 一般是 self
 - Python3.x、2.x 区别: Python3 可直接使用 **super().xx** 代替 super(Class, self).xx
- super()实现了对**父类**方法的重写，即父类方法原有的功能保持不变，通过在子类中定义与父类同名的方法，增加新的或者修改父类原有的功能
- super() 主要用于解决多重继承问题。直接用类名调用父类方法，在单继承中没有问题，但如果在多继承，则会涉及查找顺序（MRO）、重复调用（钻石继承）等种种问题

— super()案例

```
class A:  
    def __init__(self):  
        print('I am A')
```

```
class B:  
    def __init__(self):  
        print('I am B')
```

```
class C(A, B):  
    def __init__(self):  
        super().__init__() ➡ super(C, self).__init__()
```

```
c = C()
```

I am A

— super()案例

```
class A:  
    def test(self):  
        print('A', self)
```

```
class B:  
    def test(self):  
        print('B', self)
```

```
class C(A, B):  
    def __init__(self):  
        super().test()  
        super(C, self).test()  
        super(A, self).test()
```

```
C()
```

```
A <__main__.C object at 0x0000020585EA3BB0>
```

```
A <__main__.C object at 0x0000020585EA3BB0>
```

```
B <__main__.C object at 0x0000020585EA3BB0>
```

— super()

- super()和类均可调用父类实例方法，区别在于**super()后跟的方法不需要传self参数**，父类调用实例方法，第一个参数需要传self(方法的绑定)

```
class Animal:
    def __init__(self, food):
        self.food = food

    def eat(self):
        print(f"They love {self.food}.")
```

```
class Bird(Animal):
    def __init__(self, name):
        self.name = name

    def swim(self):
        print(f"{self.name} swims fast.")
```

```
class Penguin(Bird):
    def __init__(self, name, food):
        super().__init__(name)
        Animal.__init__(self, food)
```

```
peggy = Penguin("XB", "fish")
peggy.swim()
peggy.eat()
```

XB swims fast.
They love fish.

— super()

- 全部采用super()调用，与使用类调用的代码有所区别

```
class Animal:
    def __init__(self, food):
        self.food = food

    def eat(self):
        print(f"They love {self.food}.")
```

```
class Bird(Animal):
    def __init__(self, name, food):
        self.name = name
        super().__init__(food)

    def swim(self):
        print(f"{self.name} swims fast.")
```

```
class Penguin(Bird):
    def __init__(self, name, food):
        super().__init__(name, food)
```

```
peggy = Penguin("XB", "fish")
peggy.swim()
peggy.eat()
```

XB swims fast.
They love fish.

— super()案例

```
class Base(object):  
    def __init__(self):  
        print("enter Base")  
        print("leave Base")
```

```
class A(Base):  
    def __init__(self):  
        print("enter A")  
        super().__init__()  
        print("leave A")
```

```
class B(Base):  
    def __init__(self):  
        print("enter B")  
        super().__init__()  
        print("leave B")
```

```
class C(A, B):  
    def __init__(self):  
        print("enter C")  
        super().__init__()  
        print("leave C")
```

```
c = C()
```

```
enter C  
enter A  
enter B  
enter Base  
leave Base  
leave B  
leave A  
leave C
```

— super()案例

```
class Goat(object):
    def __init__(self):
        self.name = "乔丹"
        print(f"{self.name}将篮球推向了全世界")

    def god_goat(self):
        print(f"{self.name}是史上最伟大的篮球运动员")

class Mamba(Goat):
    def __init__(self):
        self.name = "科比"
        print(f"{self.name}是史上最具职业精神的篮球运动员")
        super().__init__()
        super().god_goat()

    def god_mamba(self):
        print(f"{self.name}是我最喜欢的篮球运动员")
```

```
class KingJames(Mamba):
    def __init__(self):
        self.name = "詹姆斯"
        print(f"{self.name}是史上最全能的篮球运动员")

    def god(self):
        print(f"{self.name}为三个城市带来总冠军")
        super().__init__()
        super().god_mamba()

kj = KingJames()
kj.god()
```

詹姆斯是史上最全能的篮球运动员
詹姆斯为三个城市带来总冠军
科比是史上最具职业精神的篮球运动员
乔丹将篮球推向了全世界
乔丹是史上最伟大的篮球运动员
乔丹是我最喜欢的篮球运动员

- 解题关键：各类中self指的是哪个实例

■ 类的 Mixin 设计模式: Python的接口

```
class Vehicle: # 交通工具
    def fly(self):
        # 飞行功能相应代码
        print("I am flying", self)
```

- Car类继承Vehicle类, 但Car正常无飞行功能
- 如民航飞机类直升飞机类重写fly, 则代码冗余

```
class CivilAircraft(Vehicle): # 民航飞机
    pass
```

```
class Helicopter(Vehicle): # 直升飞机
    pass
```

```
class Car(Vehicle): # 汽车并不会飞, 但按照上述继承关系, 汽车也能飞了
    pass
```

■ 类的 Mixin 设计模式

- 一种设计模式
- **比喻**：飞行，只是飞机作为交通工具的一种 **(增强) 属性**，可以为这个飞行的功能单独定义一个(增强)类，称之为 Mixin 类
- 为了让其他开发者直观地知道这是Mixin类，一般要求遵循规范，在类名末尾加上 Mixin
- **Python语法上无接口一说**

```
class Vehicle: # 交通工具
    pass
```

```
class FlyableMixin:
    def fly(self):
        # 飞行功能相应的代码
        print("I am flying", self)
```

```
class CivilAircraft(FlyableMixin, Vehicle): # 民航飞机
    pass
```

```
class Helicopter(FlyableMixin, Vehicle): # 直升飞机
    pass
```

```
class Car(Vehicle): # 汽车
    pass
```

采用某种规范（如命名规范）来解决具体的问题是python惯用的套路

■ Java中类的 Mixin实现：接口

```
// 抽象基类：交通工具类
public abstract class Vehicle {
}

// 接口：飞行器
public interface Flyable {
    public void fly()
}

// 类：实现了飞行器接口的类，在该类中实现具体的fly方法，
// 这样下面民航飞机与直升飞机在实现fly时直接重用即可
public class FlyableImpl implements Flyable {
    public void fly() {
        System.out.println("I am flying")
    }
}
```

```
// 民航飞机，继承自交通工具类，并实现了飞行器接口
public class CivilAircraft extends Vehicle implements Flyable {
    private Flyable flyable

    public CivilAircraft() {
        flyable = new FlyableImpl()
    }

    public void fly() {
        flyable.fly()
    }
}
```

```
// 直升飞机，继承自交通工具类，并实现了飞行器接口
public class Helicopter extends Vehicle implements Flyable {
    private Flyable flyable

    public Helicopter() {
        flyable = new FlyableImpl()
    }

    public void fly() {
        flyable.fly()
    }
}
```

```
// 汽车，继承自交通工具类，
public class Car extends Vehicle {
}
```



■ Mixin类实现多继承需遵循的原则

- 责任明确：必须表示某一种功能，而不是某个物品(Python 对于Mixin类的命名方式一般以 Mixin, able, ible 为后缀)；
- 功能单一：若有多个功能，则添加多个Mixin类；
- 绝对独立：不能依赖于子类的实现，子类即便没有继承这个Mixin类，也照样可以工作，只是缺少了某个功能。

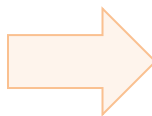
■ 不使用Mixin的弊端

- 结构复杂：多继承一个类有多个父类，父类又有多个父类，继承关系复杂
- 优先顺序模糊：多个父类中如有同名方法，容易造成混乱，子类不知道该继承哪个父类，从而增加开发难度
- 功能冲突：多重继承有多个父类，但子类只能继承一个同名方法，从而导致另一个父类的方法失效

■ 类的多态 (Polymorphism)



- Python中，不同的对象调用同一个接口(同名的方法)，从而表现出不同状态
- Python是动态语言，使用变量无需为其指定具体类型，故多态是其原始特性
- **多态发生的条件：**
 - 继承：发生在父类和子类间
 - 重写：子类重写父类的方法



```
class Duck:
    def who(self):
        print("I am a duck")
```

```
class Dog:
    def who(self):
        print("I am a dog")
```

```
class Cat:
    def who(self):
        print("I am a cat")
```

```
duck = Duck()
dog = Dog()
cat = Cat()
duck.who()
dog.who()
cat.who()
```



■ 类的多态 (Polymorphism)



— 多态发生的条件:

- 继承: 发生在父类和子类间
- 重写: 子类重写父类的方法



— 多态的作用:

- 增加程序的灵活性
- 增加程序的可扩展性

```
class Payment:
    def pay(self, money):
        print(f"通用方式支付:{money}元")
```

```
class AliPay(Payment):
    def pay(self, money):
        print(f"支付宝支付:{money}元")
```

```
class WechatPay(Payment):
    def pay(self, money):
        print(f"微信支付:{money}元")
```

```
class ApplePay(Payment):
    def pay(self, money):
        print(f"苹果支付:{money}元")
```

```
def scan(obj, money):
    obj.pay(money)
```

```
scan(AliPay(), 300)
scan(ApplePay(), 500)
```

■ 类的多态 (Polymorphism)

— 多态的约束性方法一： 通过抛异常的方式

```
class ApplePay(Payment):  
    def pay(self, money):  
        print(f'苹果支付:{money}元')
```

```
def paying(self, money):  
    print(f'苹果支付:{money}元')
```



```
class Payment:  
    def pay(self, money):  
        raise "子类没有实现正确的pay方法"
```

```
class Payment:  
    def pay(self, money):  
        raise NotImplementedError
```

```
class AliPay(Payment):  
    def pay(self, money):  
        print(f"支付宝支付:{money}元")
```

```
class WechatPay(Payment):  
    def pay(self, money):  
        print(f"微信支付:{money}元")
```

```
class ApplePay(Payment):  
    def pay(self, money):  
        print(f"苹果支付:{money}元")
```

```
def scan(obj, money):  
    obj.pay(money)
```

```
scan(AliPay(), 300)  
scan(ApplePay(), 500)
```

■ 类的多态 (Polymorphism)

— 多态的约束性方法二: 通过抽象基类和抽象方法

```
class Payment:
    def pay(self, money):
        raise "子类没有实现正确的pay方法"

from abc import abstractmethod, ABCMeta

class Payment(metaclass=ABCMeta):
    @abstractmethod
    def pay(self, money):
        pass
```

```
from abc import abstractmethod, ABCMeta
```

```
class Payment(metaclass=ABCMeta):
    @abstractmethod
    def pay(self, money):
        pass
```

```
class AliPay(Payment):
    def pay(self, money):
        print(f'支付宝支付:{money}元')
```

```
class WechatPay(Payment):
    def pay(self, money):
        print(f'微信支付:{money}元')
```

```
class ApplePay(Payment):
    def pay(self, money):
        print(f'苹果支付:{money}元')
```

```
def scan(obj, money):
    obj.pay(money)
```

```
scan(AliPay(), 300)
scan(ApplePay(), 500)
```

■ 抽象基类ABC(Abstract Base Class)

- **包含抽象方法的类，是特殊类，只能被继承，不能被实例化，且子类必须实现抽象方法**
 - 如果类是从一众对象(实例)中抽象而来，则抽象类是从一堆类中抽象而成
 - 如香蕉类、桃子类，可抽象出水果类，实例化出具体的香蕉、桃子，但水果类无法实例化出水果
 - 抽象类的本质还是类，指的是一组类的相似性，包括数据属性和方法属性，而接口只强调方法属性的相似性
- **抽象基类的作用：协同开发、约束开发，提高代码的可读性**
 - 为模块间的调用提供清晰的接口，以最精简的方式展示出代码间的逻辑关系，让模块之间的依赖更加清晰和简单
 - 抽象基类是一个介于类和接口之间的一个概念，同时具备类和接口的部分特性，可以用来实现归一化设计，从而为程序提供了逻辑和实现解耦的能力

■ 抽象基类ABC

一切皆文件

from abc import ABCMeta, abstractmethod # 利用abc模块实现抽象类

```
class All_file(metaclass=ABCMeta):
    all_type = "file"
```

```
@abstractmethod # 定义抽象方法, 无需实现功能
def read(self):
    """子类必须定义读功能"""
    pass
```

```
@abstractmethod # 定义抽象方法, 无需实现功能
def write(self):
    """子类必须定义写功能"""
    pass
```

```
# class Txt(All_file):
```

```
#     pass
```

```
#
```

```
# t1=Txt() # 报错, 子类没有定义抽象方法
```

```
class Txt(All_file): # 子类继承抽象类, 但是必须定义read和write方法
    def read(self):
        print('文本数据的读取方法')
```

```
    def write(self):
        print('文本数据的读取方法')
```

```
class Sata(All_file): # 子类继承抽象类, 但是必须定义read和write方法
    def read(self):
        print('硬盘数据的读取方法')
```

```
    def write(self):
        print('硬盘数据的读取方法')
```

```
class Process(All_file): # 子类继承抽象类, 但是必须定义read和write方法
    def read(self):
        print('进程数据的读取方法')
```

```
    def write(self):
        print('进程数据的读取方法')
```

```
text = Txt()
```

```
sat_info = Sata()
```

```
pro_msg = Process()
```

```
# 归一化处理, 即一切皆文件的思想
```

```
text.read()
```

```
sat_info.write()
```

```
pro_msg.read()
```

```
print(text.all_type, sat_info.all_type, pro_msg.all_type)
```

```
print(type(All_file), type(ABCMeta))
```

文本数据的读取方法

硬盘数据的读取方法

进程数据的读取方法

file file file

<class 'abc.ABCMeta'> <class 'type'>



■ 抽象基类ABC(Abstract Base Class)

- 抽象基类的子类，必须实现抽象基类中的抽象方法，此即为抽象，亦为接口
- 多继承
 - 在继承抽象类时，应尽量避免多继承
- 方法的实现
 - 抽象类中，可对一些抽象方法做出基础实现，也可以不实现
- 抽象模块
 - abc 模块：@abstractmethod、ABCMeta、ABC
 - collections 模块中有一些派生自 ABC 的具体类
 - collections.abc 子模块中有一些 ABC，可用于测试一个类或实例是否提供特定接口，如它是否可哈希或是否为映射等
 - numbers 库定义了跟数字对象(整数、浮点数、有理数等)有关的基类
 - io 库定义了跟I/O操作相关的基类



■ 判断类实例的函数

— type() 和 isinstance() 函数的区别

- isinstance(obj, cls): 判断obj 是否是 cls类或者其子类的实例(考虑继承关系)
- type(obj): 获取实例obj的类型, 不考虑继承关系

■ 与__class__作用相同

■ 知识拓展: **type()函数既可以返回一个对象的类型, 也可以创建出新的类型**

type()函数创建类, 参考后面元类metaclass相关知识点

■ 元类 metaclass

— 创建类的类（元类）

元类广泛应用于枚举、日志、接口检查、自动委托、自动特征属性创建、代理、框架以及自动资源锁定/同步等各类事务中

- 默认的元类：type
- 要创建一个class对象，type()函数依次传入3个参数：
 - › class的名称
 - › class继承的父类元组，如只有一个父类，注意tuple单元素写法
 - › class属性、方法构成的字典

```
def func(self): pass
```

```
Foo = type("Foo", (object,), {"count": 0, "func": func})
```

可省略object类
不能省略空元组

■ 元类 metaclass

— 通过继承type创建元类

- 通过继承type类，可修改创建类的方式(override)
- 通过metaclass=声明类的创建方式(指定元类)
- 使用元类创建类时，其__new__()方法会自动执行，用来修改新建类
- type类继承自object类

```
class ModelMeta(type):
    def __init__(cls, *args, **kwargs):
        super().__init__(*args, **kwargs)
        print("init a", cls)
        print("init a", cls.__class__.__name__)
```

```
class Model(metaclass=ModelMeta):
    pass    metaclass 老的语法: __metaclass__=ModelMeta
```

类Model相当于元类ModelMeta的实例

```
print(Model.__mro__)
print(ModelMeta.__mro__)
print(Model())
print(type(Model), type(ModelMeta))
```

```
init a <class '__main__.Model'>
init a ModelMeta
```

```
(<class '__main__.Model'>, <class 'object'>)
(<class '__main__.ModelMeta'>, <class 'type'>, <class 'object'>)
<__main__.Model object at 0x000001CB2FE8FE80>
<class '__main__.ModelMeta'> <class 'type'>
```



■ 元类 metaclass

- 通过继承type创建元类

```
class ModelMeta(type):  
    def __init__(cls, *args, **kwargs):  
        super().__init__(*args, **kwargs)  
        print("init a", cls)  
        print("init a", cls.__class__.__name__)
```

```
class Model(object, metaclass=ModelMeta):  
    pass
```

```
def func(self): pass
```

```
Foo = type("Foo", (object,), {"count": 0, "func": func})
```

```
Foo = ModelMeta("Foo", (), {"count": 0, "func": func})
```

■ 协议编程

— Dunders(Double UNDERscore): 被双下划线包围的属性名或方法名

□ **`__new__(cls, *args, **kwargs)`**: 构造方法, 创建并返回一个实例对象

- › `__new__`方法是一种特殊的负责创建类实例的静态方法(无需使用 `staticmethod` 装饰器修饰)
- › `__new__`方法调用一次, 就会得到一个对象
- › **`__new__`方法至少必须要有一个参数`cls`**, 代表要实例化的类, 此参数在实例化时由Python解释器自动提供
- › 继承自`object`的新式类才有`__new__`这一方法
- › **`__new__()`必须要有返回值, 返回由其调用生成的实例** (很重要), 这点在自定义`__new__`方法时要特别注意, 可以`return`父类`__new__`方法生成的实例, 或直接由`object`的`__new__`方法生成的实例, 若`__new__`方法没有正确返回当前类`cls`的实例, `__init__`方法不会被调用, 即便是父类的实例也不行

■ 协议编程

- `__new__(cls, *args, **kwargs)`

注意:

`__new__` 没有正确返回当前类 `cls` 的实例,
`__init__` 方法不会被调用, 即使是父类也不行

```
__new__ 被调用
cls: <class '__main__.B'> 1483397664288
b: <__main__.A object at 0x000001596176B610>
A: <class '__main__.A'> 1483397663328
B: <class '__main__.B'> 1483397664288
```

```
class A:
    pass
```

```
class B(A):
    def __init__(self):
        print("__init__被调用")

    def __new__(cls, *args, **kwargs):
        print("__new__被调用")
        print("cls: ", cls, id(cls))
        return object.__new__(A)
```

```
b = B()
print("b: ", b)
print("A: ", A, id(A))
print("B: ", B, id(B))
```

■ 协议编程

— `__new__(cls, *args, **kwargs)`

`__new__` 正确返回当前类 `cls` 的实例,
`__init__` 方法被调用

```
__new__ 被调用
cls:  <class '__main__.B'> 2160074773280
__init__ 被调用
b:  <__main__.B object at 0x000001F6EF08B610>
A:  <class '__main__.A'> 2160074769440
B:  <class '__main__.B'> 2160074773280
```

```
class A:
    pass
```

```
class B(A):
    def __init__(self):
        print("__init__被调用")

    def __new__(cls, *args, **kwargs):
        print("__new__被调用")
        print("cls: ", cls, id(cls))
        return object.__new__(B)
```

```
b = B()
print("b: ", b)
print("A: ", A, id(A))
print("B: ", B, id(B))
```

■ 单例模式

— 单例模式 (Singleton)

- 一种常见的设计模式，主要是指当一个类创建对象时，创建的所有对象都是同一个
- 主要解决的场景或问题：当一个全局类，需要频繁地创建与销毁实例，采用单例模式可以更好避免资源消耗。如页面缓存、WEB计数器、写文件操作、数据库的连接

— 单例模式的写法

- 总体思路：判断是否已有这个单例，如果有则返回，如果没有则创建
- 单例有多种写法：涉及的知识有装饰器、元类、`__new__()`、`__call__`、`super()`等

■ 单例模式的实现

— 单例模式的实现：使用__new__ ()

```
class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if not cls._instance:
            cls._instance = super().__new__(cls, *args, **kwargs)
            print(cls)
        return cls._instance
```

```
a = Singleton()
b = Singleton()
print(a is b)
```

```
<class '__main__.Singleton'>
True
```

■ 单例模式的实现

— 使用__new__()实现

继承单例类的子类若没有重写__new__(), 其依然是单例类

```
class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if not cls._instance:
            cls._instance = super().__new__(cls, *args, **kwargs)
            print(cls)
        return cls._instance
```

```
class MySingleton(Singleton):
    def __init__(self):
        print("Init...")
        pass
```

```
a = MySingleton()
b = MySingleton()
print(a is b)
```

```
<class '__main__.MySingleton'>
Init...
Init...
True
```

■ 单例模式的实现

- 单例模式的实现
(使用元类的方式)

```
class Singleton(type):
    _instance = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instance:
            cls._instance[cls] = super().__call__(*args, **kwargs)
            print(cls)
        return cls._instance
```

```
class MySingleton(metaclass=Singleton):
    def __init__(self):
        pass
```

```
a = MySingleton()
b = MySingleton()
print(a is b)
```

```
<class '__main__.MySingleton'>
True
```

■ 协议编程

- **`__init__(self)`**: 初始化方法, 在类创建实例对象(`__new__()`)之后自动触发该方法
`__init__()`须至少传入一个参数`self`(即`__new__()`返回的实例), `__init__()`无需返回值
- **`__del__(self)`**: 析构方法, 对应命令`del`, 即对象被垃圾回收的时候, 自动调用该方法, 以释放资源。除非有特殊要求, 一般不要重写。
- 实例被创建以后, 该实例会被自动加上`__class__`、`__dict__`两个属性:

■ 协议编程

- **__class__**: 获取对象所属的类(对象.__class__)
- **__base__**: 获取类的父类(类.__base__)
- **__bases__**: 获取类的父类元组(多继承, 类.__bases__)
- **__mro__**: 获取类继承关系的元组(类.__mro__, 也可以使用这种形式**类名.mro()**)
- **__subclasses__**: 获取子类的列表(类.__subclasses__)

■ 协议编程

- **`__dict__`**: 由对象内部所有属性名和属性值组成的字典
- **`vars()`**: 对应内部的`__dict__`实现
 - › **类名.`__dict__`** 会输出包含类中所有类属性（含方法）的字典
 - › **实例.`__dict__`** 会输出包含实例属性（含方法）的字典
 - › 子类调用的 `__dict__` 中，不会包含父类中的类属性，反之亦然
 - › **实例.`__dict__`** 可通过字典的形式修改属性值：实例.`__dict__`[属性名]="新属性值"
 - › **类.`__dict__`** 只读，不可修改属性值

■ 协议编程

- **`__dir__()`**: 返回对象所有属性与方法名的有序列表
- **`dir()`**: 其内部实现是在调用 `__dir__()` 方法的基础上, 对该方法返回的属性名和方法名实现排序的结果 (`dir()`的二次加工)
- 使用 `__dir__()` 方法和 `dir()` 函数输出的数据相同, 但**排序不同**
- `dir()`函数输出的不仅包括类中添加的属性和方法, 还将输出从父类继承得到的属性名和方法名



■ 协议编程

- **`__call__`**: 在类中重载 `()` 运算符, 从而使得该类实例能像函数一样被调用
 - › **用 `__call__()` 弥补 `hasattr()` 函数的短板**

魔法方法	含义
<code>__new__</code>	类的构造方法，在 <code>__init__</code> 之前创建对象
<code>__init__</code>	类的初始化方法，其功能是创建类对象时做初始化工作
<code>__del__</code>	析构方法，其功能是销毁对象时进行回收资源的操作
<code>__add__</code>	加法运算符 <code>+</code> ，当类对象 <code>X</code> 做例如 <code>X+Y</code> 或者 <code>X+=Y</code> 等操作，内部会调用此方法。但如果类中对 <code>__iadd__</code> 方法进行了重载，则类对象 <code>X</code> 在做 <code>X+=Y</code> 类似操作时，会优先选择调用 <code>__iadd__</code> 方法
<code>__radd__</code>	当类对象 <code>X</code> 做类似 <code>Y+X</code> 的运算时，会调用此方法
<code>__iadd__</code>	重载 <code>+=</code> 运算符，也就是说，当类对象 <code>X</code> 做类似 <code>X+=Y</code> 的操作时，会调用此方法
<code>__or__</code>	“或”运算符 <code> </code> ，如果没有重载 <code>__ior__</code> ，则在类似 <code>X Y</code> 、 <code>X =Y</code> 这样的语句中，“或”符号生效
<code>__repr__</code> ， <code>__str__</code>	格式转换方法，分别对应函数 <code>repr(X)</code> 、 <code>str(X)</code>
<code>__call__</code>	调用，类似于 <code>X(*args, **kwargs)</code> 语句
<code>__getattr__</code>	点号运算，用来获取类属性
<code>__setattr__</code>	属性赋值语句，类似于 <code>X.any=value</code>
<code>__delattr__</code>	删除属性，类似于 <code>del X.any</code>
<code>__getattribute__</code>	获取属性，类似于 <code>x.any</code>
<code>__getitem__</code>	索引运算，类似于 <code>X[key]</code> ， <code>X[i:j]</code>
<code>__setitem__</code>	索引赋值语句，类似于 <code>X[key]</code> ， <code>X[i:j]=sequence</code>
<code>__delitem__</code>	索引和分片删除
<code>__get__</code> ， <code>__set__</code> ， <code>__delete__</code>	描述符属性，类似于 <code>X.attr</code> ， <code>X.attr=value</code> ， <code>del X.attr</code>
<code>__len__</code>	计算长度，类似于 <code>len(X)</code>
<code>__lt__</code> ， <code>__gt__</code> ， <code>__le__</code> ， <code>__ge__</code> ， <code>__eq__</code> ， <code>__ne__</code>	比较，分别对应于 <code><</code> 、 <code>></code> 、 <code><=</code> 、 <code>>=</code> 、 <code>=</code> 、 <code>!=</code> 运算符。
<code>__iter__</code> ， <code>__next__</code>	迭代环境下，生成迭代器与取下一条，类似于 <code>I=iter(X)</code> 和 <code>next()</code>
<code>__contains__</code>	成员关系测试，类似于 <code>item in X</code>
<code>__index__</code>	整数值，类似于 <code>hex(X)</code> ， <code>bin(X)</code> ， <code>oct(X)</code>
<code>__enter__</code> ， <code>__exit__</code>	在对类对象执行类似 <code>with obj as var</code> 的操作之前，会先调用 <code>__enter__</code> 方法，其结果会传给 <code>var</code> ；在最终结束该操作之前，会调用 <code>__exit__</code> 方法（常用于做一些清理、扫尾的工作）

```
1 a=1
2 b="abc"
3 print(type(1))
4 print(type(int))
5 print(type(b))
6 print(type(str))
7
8 class Student:
9     pass
10
11 class MyStudent(Student):
12     pass
13     stu = Student()
14     print(type(stu))
15     print(type(Student))
16     print(int.__bases__)
17     print(str.__bases__)
18     print(Student.__bases__)
19     print(MyStudent.__bases__)
20     print(type.__bases__)
21     print(object.__bases__)
22     print(type(object))
```

■ 描述器 (Descriptor) 协议

- 实现了 `__get__()`、`__set__()`、`__delete__()` 中至少一种方法的对象，称为描述器
 - `__get__`：访问属性并返回属性的值。若访问属性不存在或不合法，则抛出对应异常
 - `__set__`：属性分配操作中调用，返回None
 - `__delete__`：控制删除操作，返回None
- 描述器让对象可自定义属性查找、存储和删除操作
- 描述器仅在作为一个类变量存储在另一个类中时才起作用，放入实例时则失效

```
class Ten:
    def __get__(self, obj, objtype=None):
        return 10
```

Ten 类是一个描述器

```
class A:
    x = 5
    y = Ten()      # Descriptor instance
```

```
a = A()
print(a.x)      5
print(a.y)      10
```

```
a.z = Ten()
print(a.z)
```

```
<__main__.Ten object at 0x000001DE923DFEB0>
```

本例中，值10既不存储在类字典中也不存储在实例字典中，值10是在调用时才取到

■ 描述器 (Descriptor) 用途

- 描述器的主要目的是提供一个钩子, 允许存储在类变量中的对象, 控制在属性查找期间发生的情况
- 描述器通过计算返回值而非常量
- 描述器常见用途
 - 动态查找

```
import os
```

```
class DirectorySize:  
    def __get__(self, obj, objtype=None):  
        return len(os.listdir(obj.dirname))
```

```
class Directory:  
    size = DirectorySize()  
  
    def __init__(self, dirname):  
        self.dirname = dirname
```

```
s = Directory('songs')  
g = Directory('games')  
print(s.size)  
print(g.size)  
os.remove('games/chess')  
print(g.size)
```

动态查找文件夹里有多少个文件

■ 描述器 (Descriptor) 用途

— 描述器常见用途

- 动态查找
- 托管属性

```

111
Updating 'age' to 30
in set...
222
111
Updating 'age' to 40
in set...
222
{'name': 'Mary M', '_age': 30}
{'name': 'David D', '_age': 40}
333
Accessing 'age' giving 30
Updating 'age' to 31
in set...
444
David D
Accessing 'age' giving 40
40

```

```

class LoggedAgeAccess:
    def __get__(self, obj, objtype=None):
        value = obj._age
        print(f"Accessing 'age' giving {value}")
        return value

    def __set__(self, obj, value):
        print(f"Updating 'age' to {value}")
        obj._age = value
        print("in set...")

```

```

class Person:
    age = LoggedAgeAccess()

    def __init__(self, name, age):
        self.name = name
        print(111)
        self.age = age
        print(222)

    def birthday(self):
        print(333)
        self.age += 1
        print(444)

```

代码中加入多行标记
以更好理解输出逻辑

```

mary = Person('Mary M', 30)
dave = Person('David D', 40)
print(vars(mary))
print(vars(dave))
mary.birthday()
print(dave.name)
print(dave.age)

```

```

class LoggedAgeAccess:
    def __get__(self, obj, objtype=None):
        value = obj._age
        print(f"Accessing 'age' giving {value}")
        return value

    def __set__(self, obj, value):
        print(f"Updating 'age' to {value}")
        obj._age = value

```

```

class Person:
    age = LoggedAgeAccess()

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def birthday(self):
        self.age += 1

```

此例中，一个主要问题是名称 `_age` 在类 `LoggedAgeAccess` 中属于硬耦合。这意味着每个实例只能有一个用于记录的属性（且名称不可更改）

```

mary = Person('Mary M', 30)
dave = Person('David D', 40)
print(vars(mary))
print(vars(dave))
mary.birthday()
print(dave.name)
print(dave.age)

```

```

Updating 'age' to 30
Updating 'age' to 40
{'name': 'Mary M', '_age': 30}
{'name': 'David D', '_age': 40}
Accessing 'age' giving 30
Updating 'age' to 31
David D
Accessing 'age' giving 40
40

```

■ 描述器：自动命名通告(Automatic name notification)

- 创建新类时，元类 `type` 将扫描新类的字典。如果类属性中有描述器，并且描述器中定义了 `__set_name__(self, owner, name)` 方法，则使用以下两个参数调用该方法：
 - `owner` 是使用描述器的类
 - `name` 是分配给描述器的类变量名
- 由于 `__set_name__(self, owner, name)` 方法的更新逻辑在 `type.__new__()` 中，因此该方法仅在创建类时发生，之后如果将描述器再添加到类中，则 `__set_name__()` 方法不会被自动调用，需要手动调用它

■ 描述器 (Descriptor) 用途

一 描述器常见用途

- 动态查找
- 托管属性
- 定制名称

```
{'public_name': 'name', 'private_name': '_name'}
{'public_name': 'age', 'private_name': '_age'}
{'__module__': '__main__', 'name': <__main__
.LoggedAgeAccess object at 0x000001DE2ABDFFD0>,
'age': <__main__.LoggedAgeAccess object at
0x000001DE2ABDFEE0>, '__init__': <function
Person.__init__ at 0x000001DE2AC0D090>,
'birthday': <function Person.birthday at
0x000001DE2AC0C9D0>, '__dict__': <attribute
'__dict__' of 'Person' objects>, '__weakref__':
<attribute '__weakref__' of 'Person' objects>,
'__doc__': None}
```

```
Updating 'name' to 'Peter P'
Updating 'age' to 10
Updating 'name' to 'Catherine C'
Updating 'age' to 20
{'_name': 'Peter P', '_age': 10}
{'_name': 'Catherine C', '_age': 20}
```

```
class LoggedAgeAccess:
    def __set_name__(self, owner, name):
        self.public_name = name
        self.private_name = "_" + name

    def __get__(self, obj, objtype=None):
        value = getattr(obj, self.private_name)
        print(f"Accessing {self.public_name !r} giving {value !r}")
        return value

    def __set__(self, obj, value):
        print(f"Updating {self.public_name !r} to {value !r}")
        setattr(obj, self.private_name, value)
```

```
class Person:
    name = LoggedAgeAccess()
    age = LoggedAgeAccess()

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def birthday(self):
        self.age += 1

print(vars(vars(Person)["name"]))
print(vars(vars(Person)["age"]))
print(vars(Person))
pete = Person("Peter P", 10)
kate = Person("Catherine C", 20)
print(vars(pete))
print(vars(kate))
```

本例中，*Person*类具有两个描述器实例 *name* 和 *age*。当类 *Person* 被定义时，它回调了 *LoggedAccess* 中的 *__set_name__()* 来修改字段的名称，让每个描述器拥有自己的 *public_name* 和 *private_name*

■ 描述器 (Descriptor) 案例

— 描述器协议

- `descr.__get__(self, obj, type=None) -> value`
- `descr.__set__(self, obj, value) -> None`
- `descr.__delete__(self, obj) -> None`

— 验证器类

- 验证器是一个用于托管属性访问的描述器。在存储任何数据之前，它会验证新值是否满足各种类型和范围限制。如果不满足这些限制，它将引发异常，从源头上防止数据损坏定制名称

```
from abc import ABC, abstractmethod
```

```
class Validator(ABC):
    def __set_name__(self, owner, name):
        self.private_name = "_" + name

    def __get__(self, obj, objtype=None):
        return getattr(obj, self.private_name)

    def __set__(self, obj, value):
        self.validate(value)
        setattr(obj, self.private_name, value)

    @abstractmethod
    def validate(self, value):
        pass


class OneOf(Validator):
    def __init__(self, *options):
        self.options = set(options)

    def validate(self, value):
        if value not in self.options:
            raise ValueError(f"Expected {value!r} to be one of {self.options!r}")


class Number(Validator):
    def __init__(self, minvalue=None, maxvalue=None):
        self.minvalue = minvalue
        self.maxvalue = maxvalue

    def validate(self, value):
        if not isinstance(value, (int, float)):
            raise TypeError(f"Expected {value!r} to be an int or float")
        if self.minvalue is not None and value < self.minvalue:
            raise ValueError(
                f"Expected {value!r} to be at least {self.minvalue!r}"
            )
        if self.maxvalue is not None and value > self.maxvalue:
            raise ValueError(
                f"Expected {value!r} to be no more than {self.maxvalue!r}"
            )
```

```
class String(Validator):
    def __init__(self, minsize=None, maxsize=None, predicate=None):
        self.minsize = minsize
        self.maxsize = maxsize
        self.predicate = predicate

    def validate(self, value):
        if not isinstance(value, str):
            raise TypeError(f"Expected {value!r} to be an str")
        if self.minsize is not None and len(value) < self.minsize:
            raise ValueError(
                f"Expected {value!r} to be no smaller than {self.minsize!r}"
            )
        if self.maxsize is not None and len(value) > self.maxsize:
            raise ValueError(
                f"Expected {value!r} to be no bigger than {self.maxsize!r}"
            )
        if self.predicate is not None and not self.predicate(value):
            raise ValueError(
                f"Expected {self.predicate} to be true for {value!r}"
            )
```

```
class Component:
    name = String(minsize=3, maxsize=10, predicate=str.isupper)
    kind = OneOf("wood", "metal", "plastic")
    quantity = Number(minvalue=0)

    def __init__(self, name, kind, quantity):
        self.name = name
        self.kind = kind
        self.quantity = quantity
```

```
Component("Widget", "metal", 5) # Blocked: 'Widget' is not all uppercase
Component("WIDGET", "metle", 5) # Blocked: 'metle' is misspelled
Component("WIDGET", "metal", -5) # Blocked: -5 is negative
Component("WIDGET", "metal", "V") # Blocked: 'V' isn't a number
c = Component("WIDGET", "metal", 5) # Allowed: The inputs are valid
```

■ 数据描述器与非数据描述器

- 非数据描述器 (non-data descriptor): 只实现了`__get__`方法的对象
- 数据描述器 (data descriptor): 除`__get__`方法外, 实现了`__set__`或`__delete__`至少一种方法的对象

```
class M:
    def __init__(self):
        print("Init Class M...")
        self.x = 1

    def __get__(self, instance, owner):
        print("Get m here.")
        return self.x

    def __set__(self, instance, value):
        print("Set m here.")
        self.x = value + 1
```

```
class N:
    def __init__(self):
        print("Init Class N...")
        self.x = 1

    def __get__(self, instance, owner):
        print("Get n here.")
        return self.x
```

```
class A:
    m = M()
    n = N()

    def __init__(self, m, n):
        print("Init Class A...")
        self.m = m
        self.n = n
```

```
a = A(2, 5)
```

■ 描述器涉及的一些函数、方法及访问顺序

- **hasattr()、getattr()、setattr()、delattr()**
- **__getattribute__(self, name)**：实例属性访问拦截器，在对**类实例**属性和方法访问(无论属性和方法是否存在)时，此方法均会被无条件调用。如生成实例的类中重写了该方法，函数getattr()、vars()或者对实例字典取值（[]）均会触发该方法。该方法用途：封装属性，只提供部分访问权限
- **__getattr__(self, name)**：当调用实例属性引发 AttributeError 失败时被调用。调用__getattr__前必会调用__getattribute__，定义了__getattr__方法之后，外部函数getattr()第三个参数不再起作用（详见后面案例❶）
- **__get__**：__getattribute__方法调用描述器实例时，会在描述器中（如有）触发的方法

■ 函数 `getattr()`、`hasattr()`、`setattr()`

- **`getattr(object, name, default)`**: 如果字符串`name`是对象`object`的属性之一的名称, 则返回 `name`对应的值, 否则返回 `default` (`default`可省略)
- **`hasattr(object, name)`**: 如果字符串`name`是对象`object`的属性之一的名称, 则返回 `True`, 否则返回 `False` (此功能是通过调用 `getattr(object, name)` 看是否有 `AttributeError` 异常来实现的)
- **`setattr(object, name, value)`**: 本函数与 `getattr()` 相对应。其参数为一个对象、一个字符串和一个任意值。字符串可以为某现有属性的名称, 或为新属性。只要对象允许, 函数会将值赋给属性 (甚至可以将属性改为方法)

■ 函数 `getattr()`、`setattr()`、`hasattr()`、`delattr()`

- **`getattr(object, name[, default])`:**

- 作用：获取对象的属性值。
- 参数：
 - ⊙ `object`：要访问属性的对象。
 - ⊙ `name`：属性的字符串名称。
 - ⊙ `default`（可选）：如果属性不存在时返回的默认值。
- 返回：属性的值，如果属性不存在且未提供默认值，则抛出 `AttributeError`。
- 用途：当你不知道属性名或者属性名是动态决定的时候，可以使用 `getattr` 来获取属性值。

■ 函数 `getattr()`、`setattr()`、`hasattr()`、`delattr()`

- **`setattr(object, name, value)`:**

- 作用：设置对象的属性值。
- 参数：
 - ⊙ `object`: 要设置属性的对象。
 - ⊙ `name`: 属性的字符串名称。
 - ⊙ `value`: 要设置的值。
- 返回：无。
- 用途：当你需要动态地为对象设置属性值时，可以使用 `setattr`。

■ 函数 `getattr()`、`setattr()`、`hasattr()`、`delattr()`

- **`hasattr(object, name)`:**

- 作用：检查对象是否具有指定的属性。
- 参数：
 - ⊙ `object`: 要检查属性的对象。
 - ⊙ `name`: 属性的字符串名称。
- 返回：如果对象具有该属性，则返回 `True`；否则返回 `False`。
- 用途：在访问属性之前，可以使用 `hasattr` 来检查属性是否存在，以避免抛出 `AttributeError`。

■ 函数 `getattr()`、`setattr()`、`hasattr()`、`delattr()`

- **`delattr(object, name)`:**

- 作用：删除对象的属性。
- 参数：
 - ⊙ `object`: 要删除属性的对象。
 - ⊙ `name`: 属性的字符串名称。
- 返回：无。
- 用途：当你需要动态地删除对象的属性时，可以使用 `delattr`。这通常用于实现属性的动态管理，或者在某些情况下需要清理资源。

■ 案例

```
class Person:
    def __init__(self, name):
        self.name = name
```

创建Person对象

```
p = Person("John")
```

使用getattr获取属性

```
print(getattr(p, 'name')) # 输出: John
```

使用setattr设置属性

```
setattr(p, 'age', 30)
```

```
print(p.age) # 输出: 30
```

使用hasattr检查属性

```
print(hasattr(p, 'name')) # 输出: True
```

```
print(hasattr(p, 'age')) # 输出: True
```

使用delattr删除属性

```
delattr(p, 'name')
```

```
print(hasattr(p, 'name')) # 输出: False
```

■ 函数getattr()

```
print(vars(a))
```

```
{'x': 10}
```

注意a的属性

```
10
```

```
20
```

```
Func function...
```

```
5
```

```
AttributeError: 'A' object  
has no attribute 'y'
```

```
class A:  
    v = 5
```

```
def __init__(self, x):  
    self.x = x
```

```
def func(self):  
    return "Func function..."
```

```
a = A(10)
```

```
print(getattr(a, "x"))
```

```
print(getattr(a, "y", 20))
```

```
print(getattr(a, "func")())
```

```
print(getattr(A, "v"))
```

```
print(getattr(a, "y"))
```

相当于a.x

相当于a.y, 因不存在a.y, 故返回第三个参数作为值

相当于a.func() 常用于反射

相当于A.v, 值为5

不存在a.y也无第三个参数, 故报错

■ 方法 `__getattribute__(self, name)`

— `__getattribute__()`的Python语言实现

- 当实例调用属性或方法时(无论该属性或者方法是否存在), 均会调用`__getattribute__(self, name)`方法
- 如实例a有属性x, a.x将从a.__dict__['x']查找, 如果x为数据描述器实例, 则执行描述器中的`__get__`方法并返回值; 如果x为非数据描述器, 且a有x属性, 则返回x的值, 否则执行非数据描述器中`__get__`方法并返回值; 以上皆否, 则基于`type(a)`的mro顺序向上查找, 直至找到或报错
- 在自定义的类中重写`__getattribute__()`方法, 可拦截对象属性的所有访问, 从而达到只能访问部分权限, 实现数据封装的效果
- 在自定义的类中, 如定义了`__getattr__()`方法, 只有在执行`__getattribute__()`方法引发`AttributeError`, 或在该方法内部调用`__getattr__()`, `__getattr__()`方法才会被调用
- 在自定义类外部显式调用`__getattr__`方法, 其也会被执行

```
def find_name_in_mro(cls, name, default):
    """Emulate _PyType_Lookup() in Objects/typeobject.c"""
    for base in cls.__mro__:
        if name in vars(base):
            return vars(base)[name]
    return default
```

```
def object_getattribute(obj, name):
    """Emulate PyObject_GenericGetAttr() in Objects/object.c"""
    null = object()
    objtype = type(obj)
    cls_var = find_name_in_mro(objtype, name, null)
    descr_get = getattr(type(cls_var), "__get__", null)
    if descr_get is not null:
        if (hasattr(type(cls_var), "__set__")
            or hasattr(type(cls_var), "__delete__")):
            return descr_get(cls_var, obj, objtype) # data descriptor
    if hasattr(obj, "__dict__") and name in vars(obj):
        return vars(obj)[name] # instance variable
    if descr_get is not null:
        return descr_get(cls_var, obj, objtype) # non-data descriptor
    if cls_var is not null:
        return cls_var # class variable
    raise AttributeError(name)
```

```
def getattr_hook(obj, name):
    """Emulate slot_tp_getattr_hook() in Objects/typeobject.c"""
    try:
        return obj.__getattribute__(name)
    except AttributeError:
        if not hasattr(type(obj), "__getattr__"):
            raise
        return type(obj).__getattr__(obj, name) # __getattr__
```

■ `__getattribute__`与`__getattr__`

```
class attribute...
initial...
in __getattribute__
55
in __getattribute__
in __getattr__
100
in __getattribute__
func function
66
```

```
class A:
    x = 66
    print("class attribute...")

    def __init__(self, x):
        self.x = x  # 赋值语句左边.不会调用__getattribute__
        print("initial...")

    def func(self):
        return "func function"

    def __getattr__(self, item):
        print("in __getattr__")
        return 100

    def __getattribute__(self, item):
        print("in __getattribute__")
        return super().__getattribute__(item)
```

```
a = A(10)
a.x = 55
print(a.x)
print(a.y)
print(a.func())
print(A.x)  # 类调用不会触发__getattribute__方法和__getattr__方法
```



```
class M:
    def __init__(self):
        print("Init Class M...")
        self.x = 1

    def __get__(self, instance, owner):
        print("Get m here.")
        return self.x

    def __set__(self, instance, value):
        print("Set m here.")
        self.x = value + 1

class N:
    def __init__(self):
        print("Init Class N...")
        self.x = 1

    def __get__(self, instance, owner):
        print("Get n here.")
        return self.x
```

```
class A:
    m = M()
    n = N()

    def __init__(self, m, n):
        print("Init Class A...")
        self.m = m
        self.n = n

a = A(2, 5)
print(a.__dict__)
print(A.__dict__)
print(a.n)
print(A.n)
print(a.m)
a.m = 6
print(a.m)
print("---分隔符---")
print(A.__dict__["n"].__get__(None, A))
print(A.__dict__["n"].__get__(a, A))
```

```
Init Class M...
Init Class N...
Init Class A...
Set m here.
{'n': 5}
{'__module__': '__main__',
 'm': <__main__.M object
 at 0x000002383A943FD0>,
 'n': <__main__.N object
 at 0x000002383A943DF0>,
 '__init__': <function
 A.__init__ at
 0x000002383A93C9D0>,
 '__dict__': <attribute
 '__dict__' of 'A'
 objects>, '__weakref__':
 <attribute '__weakref__'
 of 'A' objects>,
 '__doc__': None}
5
Get n here.
1
Get m here.
3
Set m here.
Get m here.
7
---分隔符---
Get n here.
1
Get n here.
1
```

```
class X:
    def __init__(self, x):
        print("class X init...")
        self.x = x

    def __get__(self, instance, owner):
        print("in __get__")
        return self.x, "get的值"

    def __set__(self, instance, value):
        print("in __set__")
        instance.__dict__["x"] = value
        print("*****")
        print(instance.__dict__["x"], "修改set值")
```

```
class A:
    x = X(0)
    print("class attribute...")

    def __init__(self, x):
        self.x = x
        print("class A's instance init...")

    def func(self):
        return "func function"

    def __getattr__(self, item):
        print("in __getattr__")
        return 100

    def __getattribute__(self, item):
        print("in __getattribute__")
        return super().__getattribute__(item)
```

定义完类便输出，此时a = A(10)还未开始执行

不是对字典赋值，因而触发__getattribute__方法

```
a = A(10)
a.x = 55
print(a.x)
print(a.y)
print(a.func())
print(A.x)
print(a.__dict__)
print(a.x)
print("a.y = ", a.y)
print(a.__dict__)
q = getattr(a, "p", 88)
print("q:", q)
print(a.__dict__)
```

```
class X init...
class attribute...

in __set__
in __getattribute__
*****
in __getattribute__
10 修改set值
class A's instance init...
in __set__
in __getattribute__
*****
in __getattribute__
55 修改set值
in __getattribute__
in __get__
(0, 'get的值')
in __getattribute__
in __getattr__
100
in __getattribute__
func function
in __get__
(0, 'get的值')
in __getattribute__
{'x': 55}
in __getattribute__
in __get__
(0, 'get的值')
in __getattribute__
in __getattr__
a.y = 100
in __getattribute__
{'x': 55}
in __getattribute__
in __getattr__
q: 100 ① 注意q的值不是88
in __getattribute__
{'x': 55}
```

- 对实例属性赋值，左边点语法不会触发__getattribute__方法，但有方括号表示实例先调用__dict__，然后对其中某个键赋值，从而触发该方法，另外vars()中传入实例作为参数也将触发该类__getattribute__()
- A.x 点操作符查找的逻辑在type.__getattribute__()中，不会执行实例调用中的object.__getattribute__(self, name)
- __getattr__()方法的优先级高于函数getattr() ①

■ 赋值语句、函数vars()触发__getattribute__

```
class A:
    def __init__(self, x):
        self.x = x

    def __getattr__(self, name):
        print("in __getattr__")
        return 100

    def __getattribute__(self, name):
        print("in __getattribute__")
        print(name)
        return super().__getattribute__(name)
```

```
a = A(10)
print("*****")
a.__dict__ = {"z": 99}
print("^^^^^")
print(vars(a))
```

^^^^^

in __getattribute__
__dict__
{'z': 99}

■ `__dir__()`与 `dir()`触发`__getattribute__`的区别

```
class A:
    def __init__(self, x):
        self.x = x

    def __getattr__(self, name):
        print("in __getattr__")
        return 100

    def __getattribute__(self, name):
        print("in __getattribute__")
        print(name)
        return super().__getattribute__(name)

a = A(10)
print("#####")
print(a.__dir__())
print("@@@@@")
print(dir(a))
```

```
#####
in __getattribute__
__dir__
in __getattribute__
__dict__
in __getattribute__
__class__
['x', '__module__', '__init__',
 '__getattr__', '__getattribute__',
 '__dict__', '__weakref__',
 '__doc__', '__new__', '__repr__',
 '__hash__', '__str__',
 '__setattr__', '__delattr__',
 '__lt__', '__le__', '__eq__',
 '__ne__', '__gt__', '__ge__',
 '__reduce_ex__', '__reduce__',
 '__subclasshook__',
 '__init_subclass__', '__format__',
 '__sizeof__', '__dir__',
 '__class__']

@@@@@
in __getattribute__
__dict__
in __getattribute__
__class__
['__class__', '__delattr__',
 '__dict__', '__dir__', '__doc__',
 '__eq__', '__format__', '__ge__',
 '__getattr__', '__getattribute__',
 '__gt__', '__hash__', '__init__',
 '__init_subclass__', '__le__',
 '__lt__', '__module__', '__ne__',
 '__new__', '__reduce__',
 '__reduce_ex__', '__repr__',
 '__setattr__', '__sizeof__',
 '__str__', '__subclasshook__',
 '__weakref__', 'x']
```

■ __getattr__陷阱

- 如右边代码所示，在自定义的__getattr__()方法中，需要注意无限循环调用的问题，即不要在该方法的代码中出现诸如self.x此类访问当前实例属性的内容，否则将出现无限循环调用的问题，应当尽量使用基类的__getattr__()或使用super()的方式来调用

```
class A(object):
    Cnum = 10

    def __init__(self, inum):
        self.inum = inum

    def __getattr__(self, name):
        print(f"正在查找{name}")
        try:
            # 这样的写法是错误的，很容易入坑
            return self.name
            # 正确的是：
            # return object.__getattr__(self, name)
            # 或者
            # return super().__getattr__(name)
        except AttributeError:
            print("没有找到: {name}")

a = A(0)
print(A.Cnum)
print(a.inum)
print(a.Cnum)
print(a.g)
```

■ 描述器：点语法调用属性的访问顺序

- 通过实例调用属性：其优先级分别是：数据描述器的优先级最高，其次是实例变量、非数据描述器、类变量，最后是 `__getattr__()`（如果存在的话）
 - 数据描述器始终会覆盖实例字典
 - 非数据描述器会被实例字典覆盖
- 通过类调用：像 `A.x` 点操作符查找的逻辑在 **`type.__getattribute__()`** 中，步骤与 **`object.__getattribute__()`** 相似，但实例字典查找改为搜索类的method resolution order
- 通过 `super()` 调用：`super` 的点操作符查找的逻辑在 `super()` 返回的对象的 `__getattribute__()` 方法中。类似 `super(A, obj).m` 的点分查找将在 `obj.__class__.__mro__` 中搜索紧接在A之后的基类B，然后返回 `B.__dict__['m'].__get__(obj, A)`，如果 `m` 不是描述器，则直接返回其值

■ 描述器应用：ORM（对象关系映射）

- ORM核心思路是：将数据存储在外部数据库中，Python 实例仅持有数据库表中对应的键，描述器负责对值进行查找或更新

```
import sqlite3
conn = sqlite3.connect('entertainment.db')

class Field:

    def __set_name__(self, owner, name):
        self.fetch = f'SELECT {name} FROM {owner.table} WHERE {owner.key}=?;'
        self.store = f'UPDATE {owner.table} SET {name}=? WHERE {owner.key}=?;'

    def __get__(self, obj, objtype=None):
        return conn.execute(self.fetch, [obj.key]).fetchone()[0]

    def __set__(self, obj, value):
        conn.execute(self.store, [value, obj.key])
        conn.commit()
```

```
class Movie:
    table = 'Movies'
    key = 'title'
    director = Field()
    year = Field()

    def __init__(self, key):
        self.key = key
```

```
class Song:
    table = 'Music'
    key = 'title'
    artist = Field()
    year = Field()
    genre = Field()

    def __init__(self, key):
        self.key = key
```

■ Property属性描述器

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def get_area(self):
        """Calculate the area of the rectangle."""
        return self.width * self.height
```

使用

```
rect = Rectangle(4, 5)
print(rect.get_area()) # 输出: 20
```



■ Property属性描述器

— @property 装饰器

- 调用属性的方式调用方法
- 通过property检查实例参数，如属性只读、参数制约等

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    @property
    def area(self):
        """Calculate the area of the rectangle."""
        return self.width * self.height

    @area.setter
    def area(self, value):
        raise AttributeError("Cannot set the area directly. Use width and height instead.")
```

```
# 使用
rect = Rectangle(4, 5)
print(rect.area) # 输出: 20
```

```
# 尝试设置area属性将会引发错误
# rect.area = 100 # 这会引发AttributeError
```

■ Property属性描述器

– @property 装饰器

- 调用属性的方式调用方法
- 通过property检查实例参数，如属性只读、参数制约等

```
class C:
    def __init__(self):
        self._x = None

    @property
    def x(self):
        """I'm the 'x' property."""
        return self._x

    @x.setter
    def x(self, value):
        self._x = value

    @x.deleter
    def x(self):
        del self._x
```

■ @property 只读属性

- 私有属性：
- @property

Python 中的 `property` 是一个装饰器，用于创建只读属性，或者在属性被访问时执行特定的操作。它允许你将方法当作属性来访问，同时还可以控制属性的访问。

■ 访问器 (getter) :

`property` 默认会创建一个只读属性，你可以通过 `fget` 参数指定一个方法作为属性的获取器 (getter)。

■ 修改器 (setter) :

如果你想要属性可写，可以提供 `fset` 参数指定一个方法作为属性的设置器 (setter)。

■ 删除器 (deleter) :

如果你想要属性可以被删除，可以提供 `fdel` 参数指定一个方法作为属性的删除器 (deleter)。

■ 文档字符串 :

`property` 对象可以有一个文档字符串，这个字符串可以通过属性的 `__doc__` 属性访问。

■ @property

```
class Person:
```

```
    def __init__(self, name):  
        self._name = name
```

```
    @property
```

```
    def name(self):  
        """The name property."""  
        return self._name
```

```
    @name.setter
```

```
    def name(self, value):  
        if not value:  
            raise ValueError("Name cannot be empty")  
        self._name = value
```

```
    @name.deleter
```

```
    def name(self):  
        raise AttributeError("Cannot delete the name attribute")
```

```
p = Person("John")  
print(p.name) # John  
p.name = "Jane" # 更新属性  
print(p.name) # Jane  
# p.name = "" # 这会抛出 ValueError  
# del p.name # 这会抛出 AttributeError
```

■ Property属性描述器

- @property 案例

```
class Student:
    def __init__(self, name, math, chinese, english):
        self.name = name
        self.math = math
        self.chinese = chinese
        self.english = english
```



```
class Student:
    def __init__(self, name, math, chinese, english):
        self.name = name
        if 0 <= math <= 100:
            self.math = math
        else:
            raise ValueError("Valid value must be in [0, 100]")

        if 0 <= chinese <= 100:
            self.chinese = chinese
        else:
            raise ValueError("Valid value must be in [0, 100]")

        if 0 <= english <= 100:
            self.english = english
        else:
            raise ValueError("Valid value must be in [0, 100]")
```

▪ `__init__`里有太多的判断逻辑，影响代码的可读性



```
class Student:
    def __init__(self, name, math, chinese, english):
        self.name = name
        self.math = math
        self.chinese = chinese
        self.english = english
```

```
@property
```

```
def math(self):
    return self._math
```

```
@math.setter
```

```
def math(self, value):
    if 0 <= value <= 100:
        self._math = value
    else:
        raise ValueError("Valid value must be in [0, 100]")
```

使用property检查参数

```
class Student:
    def __init__(self, name, math, chinese, english):
        self.name = name
        self.math = math
        self.chinese = chinese
        self.english = english

    @property
    def math(self):
        return self._math

    @math.setter
    def math(self, value):
        if 0 <= value <= 100:
            self._math = value
        else:
            raise ValueError("Valid value must be in [0, 100]")
```

▪ 这样的代码会出现什么问题？

各门成绩，每一次检验的方法都一样，所以最好有方法可以批量实现这件事，只写一次if判断



```
class Score:
    def __init__(self, default=0):
        self._score = default

    def __set__(self, instance, value):
        if not isinstance(value, int):
            raise TypeError('Score must be integer')
        if not 0 <= value <= 100:
            raise ValueError('Valid value must be in [0, 100]')

        self._score = value

    def __get__(self, instance, owner):
        return self._score

    def __delete__(self):
        del self._score
```

代码改进方案

当访问一个属性时，不直接给一个值，而是给一个描述器，让访问和修改设置时自动调用 `__get__` 方法和 `__set__` 方法。再在 `__get__` 方法和 `__set__` 方法中进行某种逻辑处理，便可实现更改操作属性的目的。这就是描述器做的事情。

```
class Student:
    math = Score(0)
    chinese = Score(0)
    english = Score(0)

    def __init__(self, name, math, chinese, english):
        self.name = name
        self.math = math
        self.chinese = chinese
        self.english = english
```

■ 属性描述器Property代码实现

- property()函数（其实是类）：返回 property 属性
 - class property(fget=None, fset=None, fdel=None, doc=None)

```
class C:
    def __init__(self):
        self._x = None

    def getx(self):
        return self._x

    def setx(self, value):
        self._x = value

    def delx(self):
        del self._x

x = property(getx, setx, delx, "I'm the 'x' property.")
```

```

class Property:
    "Emulate PyProperty_Type() in Objects/descrobject.c"

    def __init__(self, fget=None, fset=None, fdel=None, doc=None):
        self.fget = fget
        self.fset = fset
        self.fdel = fdel
        if doc is None and fget is not None:
            doc = fget.__doc__
        self.__doc__ = doc
        self._name = ''

    def __set_name__(self, owner, name):
        self._name = name

    def __get__(self, obj, objtype=None):
        if obj is None:
            return self
        if self.fget is None:
            raise AttributeError(f"property '{self._name}' has no getter")
        return self.fget(obj)

    def __set__(self, obj, value):
        if self.fset is None:
            raise AttributeError(f"property '{self._name}' has no setter")
        self.fset(obj, value)

    def __delete__(self, obj):
        if self.fdel is None:
            raise AttributeError(f"property '{self._name}' has no deleter")
        self.fdel(obj)

    def getter(self, fget):
        prop = type(self)(fget, self.fset, self.fdel, self.__doc__)
        prop._name = self._name
        return prop

    def setter(self, fset):
        prop = type(self)(self.fget, fset, self.fdel, self.__doc__)
        prop._name = self._name
        return prop

    def deleter(self, fdel):
        prop = type(self)(self.fget, self.fset, fdel, self.__doc__)
        prop._name = self._name
        return prop

```

```

class Cell:
    ...

    @property
    def value(self):
        "Recalculate the cell before returning value"
        self.recalc()
        return self._value

```

■ Functions and methods代码实现

```
class MethodType:
    "Emulate PyMethod_Type in Objects/classobject.c"

    def __init__(self, func, obj):
        self.__func__ = func
        self.__self__ = obj

    def __call__(self, *args, **kwargs):
        func = self.__func__
        obj = self.__self__
        return func(obj, *args, **kwargs)
```

```
class Function:
    ...

    def __get__(self, obj, objtype=None):
        "Simulate func_descr_get() in Objects/funcobject.c"
        if obj is None:
            return self
        return MethodType(self, obj)
```

■ 静态方法的Python实现

```
class StaticMethod:
    """Emulate PyStaticMethod_Type() in Objects/funcobject.c"""

    def __init__(self, f):
        self.f = f

    def __get__(self, obj, objtype=None):
        return self.f

    def __call__(self, *args, **kwargs):
        return self.f(*args, **kwargs)

class E:
    @staticmethod
    def f(x):
        return x * 10
```

■ 类方法的Python实现

```
class MethodType:
```

```
    """Emulate PyMethod_Type in Objects/classobject.c"""
```

```
    def __init__(self, func, obj):
```

```
        self.__func__ = func
```

```
        self.__self__ = obj
```

```
    def __call__(self, *args, **kwargs):
```

```
        func = self.__func__
```

```
        obj = self.__self__
```

```
        return func(obj, *args, **kwargs)
```

```
class ClassMethod:
```

```
    """Emulate PyClassMethod_Type() in Objects/funcobject.c"""
```

```
    def __init__(self, f):
```

```
        self.f = f
```

```
    def __get__(self, obj, cls=None):
```

```
        if cls is None:
```

```
            cls = type(obj)
```

```
        if hasattr(type(self.f), '__get__'):
```

```
            # This code path was added in Python 3.9
```

```
            # and was deprecated in Python 3.11.
```

```
            return self.f.__get__(cls, cls)
```

```
        return MethodType(self.f, cls)
```

```
class F:
```

```
    @classmethod
```

```
    def f(cls, x):
```

```
        return cls.__name__, x
```



```
class M:
    def __init__(self):
        print("Init Class M...")
        self.x = 1

    def __get__(self, instance, owner):
        print("Get m here.")
        return self.x

    def __set__(self, instance, value):
        print("Set m here.")
        self.x = value + 1

class N:
    def __init__(self):
        print("Init Class N...")
        self.x = 1

    def __get__(self, instance, owner):
        print("Get n here.")
        return self.x
```

```
class A:
    m = M()
    n = N()

    def __init__(self, m, n):
        print("Init Class A...")
        self.m = m
        self.n = n
```

```
a = A(2, 5)
print(a.__dict__)
print(A.__dict__)
print(a.n)
print(A.n)
print(a.m)
a.m = 6
print(a.m)
print("---分隔符---")
print(A.__dict__["n"].__get__(None, A))
print(A.__dict__["n"].__get__(a, A))
```

- 描述器可以直接以 desc.__get__(obj) 或 desc.__get__(None, cls) 调用
- 更常见的是通过属性访问自动调用描述器。如 a.x 找到一个描述器，则将以 desc.__get__(a, type(a)) 调用

```
Init Class M...
Init Class N...
Init Class A...
Set m here.
{'n': 5}
{'__module__': '__main__',
 'm': <__main__.M object
 at 0x000002383A943FD0>,
 'n': <__main__.N object
 at 0x000002383A943DF0>,
 '__init__': <function
 A.__init__ at
 0x000002383A93C9D0>,
 '__dict__': <attribute
 '__dict__' of 'A'
 objects>, '__weakref__':
 <attribute '__weakref__'
 of 'A' objects>,
 '__doc__': None}
5
Get n here.
1
Get m here.
3
Set m here.
Get m here.
7
---分隔符---
Get n here.
1
Get n here.
1
```

■ 描述器 (Descriptor) 用途

一 描述器常见用途

- 动态查找
- 托管属性
- 定制名称

在属性查找期间，描述器由点运算符调用。如果使用 `vars(some_class)[descriptor_name]` 间接访问描述器，则返回描述器实例而不调用它

```
{'public_name': 'name', 'private_name': '_name'}
{'public_name': 'age', 'private_name': '_age'}
{'__module__': '__main__', 'name': <__main__.LoggedAgeAccess object at 0x000001DE2ABDFFD0>,
'age': <__main__.LoggedAgeAccess object at 0x000001DE2ABDFEE0>, '__init__': <function
Person.__init__ at 0x000001DE2AC0D090>,
'birthday': <function Person.birthday at 0x000001DE2AC0C9D0>, '__dict__': <attribute
'__dict__' of 'Person' objects>, '__weakref__':
<attribute '__weakref__' of 'Person' objects>,
'__doc__': None}
```

```
Updating 'name' to 'Peter P'
Updating 'age' to 10
Updating 'name' to 'Catherine C'
Updating 'age' to 20
{'_name': 'Peter P', '_age': 10}
{'_name': 'Catherine C', '_age': 20}
```

```
class LoggedAgeAccess:
    def __set_name__(self, owner, name):
        self.public_name = name
        self.private_name = "_" + name

    def __get__(self, obj, objtype=None):
        value = getattr(obj, self.private_name)
        print(f"Accessing {self.public_name !r} giving {value !r}")
        return value

    def __set__(self, obj, value):
        print(f"Updating {self.public_name !r} to {value !r}")
        setattr(obj, self.private_name, value)
```

```
class Person:
    name = LoggedAgeAccess()
    age = LoggedAgeAccess()

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def birthday(self):
        self.age += 1
```

```
print(vars(vars(Person)["name"]))
print(vars(vars(Person)["age"]))
print(vars(Person))
pete = Person("Peter P", 10)
kate = Person("Catherine C", 20)
print(vars(pete))
print(vars(kate))
```

本例中，*Person* 类具有两个描述器实例 *name* 和 *age*。当类 *Person* 被定义时，它回调了 *LoggedAccess* 中的 *__set_name__()* 来修改字段的名称，让每个描述器拥有自己的 *public_name* 和 *private_name*

■ 描述器

- 描述器的机制嵌入在 `object`, `type` 和 `super()` 的 `__getattribute__()` 方法中, 类从 `object`, `type` 或 `super()` 继承此机制
- 数据和非数据描述器的不同之处在于, 如何计算实例字典中属性的替代值。如果实例的字典具有与数据描述器实例同名的属性, 则数据描述器优先; 如果实例字典具有与非数据描述器同名的属性, 则该实例字典属性优先
- 由于描述器的逻辑在 `__getattribute__()` 中, 故重写该方法会阻止描述器的自动调用
- `object.__getattribute__()`和`type.__getattribute__()`会用不同的方式调用`__get__()`, 前者会传入实例, 也可以包括类, 后者传入的实例为 `None`, 并且总是包括类
- 为使数据描述器成为只读, 可同时定义 `__get__()` 和 `__set__()`, 并在 `__set__()` 中引发 `AttributeError`。用引发异常的占位符定义 `__set__()` 方法使其成为数据描述器【经验】

■ 描述器 (Descriptor) 属性访问顺序

- 属性访问的默认行为是从一个对象的字典中获取、设置或删除属性。对于实例来说, `a.x` 的查找顺序会从 `a.__dict__['x']` 开始, 然后是 `type(a).__dict__['x']`, 接下来依次查找 `type(a)` 的方法解析顺序 (MRO)。如果找到的值是定义了某个描述器方法的对象, 则 Python 可能会重写默认行为并转而发起调用描述器方法。具体发生在优先级链的哪个环节则要根据所定义的描述器方法及其被调用的方式来决定
- 描述器是一个强大而通用的协议, 是属性、方法、静态方法、类方法和 `super()` 背后的实现机制, 在 Python 内部被广泛使用。描述器简化了底层的 C 代码并为 Python 的日常程序提供了一组灵活的新工具